UNIVERSITY OF MACEDONIA
SCHOOL OF INFORMATION SCIENCES
DEPARTMENT OF APPLIED INFORMATICS

AstroCluster: A Framework Agnostic Clustering Tool for Semantic Analysis of Projects and Technical Debt

Bachelor Thesis

of

Kostandin Kote

Thessaloniki, February, 2025

Abstract

Several studies have explored the world of code analysis using neural networks and graph-based methods. These methods, while offering robust insights into code structure and functionality, lack extensive use cases for finding similarity between code files and their semantic analysis in different projects. Despite these advancements, the challenge of understanding the semantic relationships between files in large, diverse codebases remains underexplored. Identifying these relationships is critical for tasks such as refactoring, modularization, and assessing technical debt, which require a deep understanding of the intent and context of the code. This thesis introduces a framework-agnostic clustering model for organizing software project files based on contextual semantic similarity. The model utilizes embeddings generated by a fine-tuned UniXcoder embedder to represent file content, which are then clustered using a similarity metric and clustering paradigm. Users can manually inspect clusters, enabling tailored refactoring and automatic technical debt analysis.

Keywords: neural networks, clustering algorithms, embedders, framework-agnostic, refactoring, technical debt, similarity metric, interest, modularization, codebases, semantic relationships, UniXcoder

Acknowledgements

First, I owe my gratitude to my family and parents for always supporting me in this journey. Their love and understanding helped me focus and try to strive to be a better person every day.

Next, I want to thank my close friends for being there when I needed them the most. Their companionship helped me clear my mind when getting stuck on problems and allowed me to find solutions to problems in a crystal-clear state of mind.

And finally, I want to show my deepest appreciation to Professor A. Ampatzoglou and supervisor N. Nikolaidis for aiding me in my research for this topic and discussing different approaches so that we could be able to create this tool.

Contents

1 Introduction

The rapid growth of software development has led to increasingly complex codebases, making it challenging to maintain, refactor, and analyze projects effectively. Understanding the relationships between files within a project and their semantic meaning is vital for improving code quality, reducing technical debt, and enabling efficient development practices. While traditional methods of code analysis, such as static analysis tools, have proven useful for detecting syntax errors and enforcing style guidelines, they often fall short in capturing the deeper semantic relationships between code components.

Recent advancements in machine learning, particularly the use of neural networks and graph-based methods, have opened new avenues for code analysis. These approaches provide robust insights into code structure and functionality, facilitating tasks such as code summarization, defect detection, and dependency analysis. However, they often fail to address the problem of semantic similarity between files, particularly across diverse frameworks and project structures. The lack of a generalized solution for organizing and understanding code files in terms of their semantic content leaves a critical gap in modern code analysis techniques.

This thesis aims to bridge this gap by introducing a framework-agnostic clustering model that organizes project files based on their semantic similarity. At the core of this approach is UniXcoder, a state-of-the-art neural model fine-tuned to generate embeddings that represent the semantic content of code. These embeddings serve as input to a clustering algorithm that groups files into meaningful clusters, enabling developers to gain a high-level understanding of the project's structure and relationships. By providing a mechanism for users to manually inspect clusters, the model allows developers to identify areas for refactoring, modularization, and technical debt remediation.

The framework-agnostic nature of the proposed model ensures its applicability across a wide range of programming environments and ecosystems. In this version, it can analyze various types of Java projects, making it a versatile tool for diverse software development scenarios. This flexibility not only enhances its practical utility but also ensures relevance in real-world development workflows.

This introduction lays the foundation for an approach that integrates semantic analysis, clustering algorithms, and machine learning to address a pressing need in software engineering. By exploring this tool's design, implementation, and applications, this thesis seeks to contribute to the growing field of code analysis tools while providing practical insights for improving code maintainability, quality and technical debt.

Section 2 discusses related work in the realm of code analysis. Section 3 briefly addresses the methodologies and tools used, while Sections 4 and 5 focus on the development process of the model and tool. Section 6 highlights the tool's usefulness for calculating technical debt, followed by Section 7, where clustering results for projects like Apache Hive and Apache Kafka are analyzed. Finally, Section 8 concludes the thesis, offering insights and proposing ideas for the tool's future development.

2 Related Work

Static analysis of code using machine learning and deep learning techniques has been a wide area of research over the years, with a vast range of use cases (classification, clustering, code representation and similarity). Code structure is usually the basis for all these use cases since it can be represented using an AST (Abstract Syntax Tree), which in turn works as the entry point for static code analysis.

A study proposed by Miltiadis Allamanis, Marc Brockschmid and Mahmoud Khademi [1] suggests that using graphs and graph-based deep learning to capitalize of the unique opportunities code provides for semantic analysis, opposed to traditional NLP techniques, performs better on predicting the names of the variables given a context and the correct selection of such names. This is achieved by using structural properties of code such as the imports of a program. They also note that via testing, the model was able to identify several bugs in mature projects, proving the usefulness of such graphs.

Ben-Nun, Jakobovits, and Hoefler (2018) [5] extend the discussion of code semantics with their work on inst2vec, an embedding space based on an Intermediate Representation (IR) of code that is agnostic to the programming language. Unlike traditional syntactic representations, their approach leverages both the data- and control-flow inherent to the program, defining a contextual flow for the IR that captures the robust semantics of code. By applying this representation to diverse program analysis tasks—such as performance prediction, compute device mapping, and algorithm classification—their method achieves state-of-the-art results without task-specific fine-tuning. This work demonstrates the potential of embeddings to bridge human- and machine-generated programs, creating a unified representation that performs effectively across various domains.

The pre-training of deep bidirectional transformers, as introduced in the BERT model by Kenton and Toutanova (2019) [9], further highlights the significance of contextual understanding in representation learning. Though originally designed for natural language processing tasks, BERT's ability to jointly condition on both left and right contexts in all layers provides a foundation for applying such techniques to source code analysis. They also note that fine-tuned BERT models have achieved state-of-the-art results across multiple tasks, including question answering and language inference, showcasing the versatility of transformer-based architectures. This approach has inspired adaptations for code, where the structural parallels between code and natural language benefit from BERT's deep contextual embeddings, enabling precise semantic comprehension and task-specific performance improvements.

3 Methodologies and Tools

3.1 Java

Java is a widely used high-level object-oriented programming language. It is a general-purpose programming language designed to allow developers to write once run anywhere, meaning that compiled Java code can run on all platforms that support Java. This feature is achieved through the Java Runtime Environment (JRE) and the Java Virtual Machine (JVM), which together enable cross-platform functionality.

Java was originally developed by James Gosling and his team at Sun Microsystems, and it was first released in 1995. The language has since evolved through various versions, each introducing new features and enhancements to improve performance, security, and usability. The main benefits of Java include but are not limited to:

- **Object-Oriented**: Java follows the object-oriented programming paradigm, which organizes software design around data, or objects. This approach promotes greater modularity and code reusability, especially in bigger projects.

- **Platform Independence**: Java code is compiled into bytecode, which can be executed on any device equipped with the JVM as mentioned above.

- **Standard Library Support**: Java includes a plethora of standard library APIs which provide a wide range of functionalities such as data structures, networking, graphical user interface utilities, multithreading and much more.

- **Automatic Memory Management**: Java includes an automatic garbage collection mechanism to manage memory allocation and deallocation, reducing the probability of memory leaks.

Java is used as the primary language for the backend (business logic).

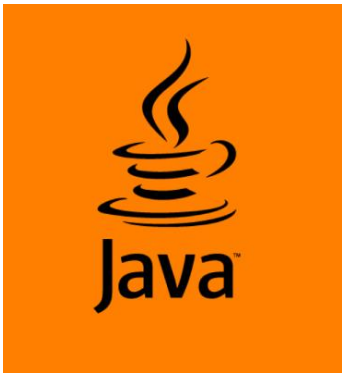


Figure 1: Java Programming Language Logo

3.2 Python and Google Colab

Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation.

Python is dynamically typed, and garbage collected. It supports multiple programming paradigms, including structured (particularly procedural), object-oriented and functional programming. Just as with Java, Python offers a wide range of libraries out of the box that assist in developer productivity.

Nowadays, the Python ecosystem has increasingly centered around the field of machine learning and artificial intelligence. This evolution is driven by Python's versatility, simplicity, and the extensive range of powerful libraries and frameworks available for data science and AI development. Key libraries such as TensorFlow, PyTorch, scikit-learn, and Keras have become essential tools for researchers and developers. This fact makes the language ideal for implementing machine learning and specifically deep learning solutions since its simple API hides all the implementation complexity of these libraries that are originally written in C++ for performance reasons.

Additionally, what makes Python ideal for machine learning and deep learning tasks are:

- **The Robust Community Support and Comprehensive Documentation**: Python has an active community of developers and researchers who contribute to resources, tutorials, and libraries. This support network makes it easier to find solutions to problems, collaborate on projects, and stay updated with the latest advancements.

- **Google Colab Support**: Google Colab is a free cloud service that provides Jupyter notebooks with free access to computational resources, including GPUs. This makes it easier for developers and researchers to prototype, test, and share their machine learning models without substantial hardware limitations.

Python and Google Colab are used in the backend (gRPC cluster server, clustering model) and in the fine-tuning process of the model.

Figure 2: Python Programming Language Logo

3.3 TypeScript

TypeScript is a free and open-source high-level programming language developed by Microsoft that adds static typing with optional type annotations to JavaScript. It is designed for the development of large applications and transpiles to JavaScript.

TypeScript may be used to develop JavaScript applications for both client-side and server-side execution (as with Node.js, Deno or Bun). Multiple options are available for transpilation. The default TypeScript Compiler can be used, or the Babel compiler can be invoked to convert TypeScript to JavaScript. In this research, TypeScript will be used as the main technology for the client-side.

TypeScript supports definition files that can contain type information of existing JavaScript libraries, much like C++ header files can describe the structure of existing object files. This enables other programs to use the values defined in the files as if they were statically typed TypeScript entities.

The TypeScript compiler is itself written in TypeScript and compiled to JavaScript. It is licensed under the Apache License 2.0. Anders Hejlsberg, lead architect of C# and creator of Delphi and Turbo Pascal, has worked on the development of TypeScript.

The advantages of TypeScript over JavaScript include:

- **Static Typing**: TypeScript introduces static types, which help catch errors at compile time rather than at runtime, leading to more robust and maintainable code.

- **Better Code Readability and Maintainability**: With explicit types and interfaces, TypeScript makes the code more readable and easier to understand, especially in large codebases.

- **Enhanced Refactoring Capabilities**: The type system facilitates safer and more efficient refactoring, making it easier to manage and update large codebases.

TypeScript is used as the main language for the frontend (client).

Figure 3: TypeScript Programming Language Logo

3.4 PostgreSQL

PostgreSQL is a powerful, open-source object-relational database system that uses and extends the SQL language combined with many features that safely store and scale the most complicated data workloads. The origins of PostgreSQL date back to 1986 as part of the POSTGRES project at the University of California at Berkeley and has more than 35 years of active development on the core platform.

PostgreSQL has earned a strong reputation for its proven architecture, reliability, data integrity, robust feature set, extensibility, and the dedication of the open-source community behind the software to consistently deliver performant and innovative solutions. PostgreSQL runs on all major operating systems, has been ACID-compliant since 2001, and has powerful add-ons such as the popular PostGIS geospatial database extender. It is no surprise that PostgreSQL has become the open-source relational database of choice for many people and organizations.

PostgreSQL comes with many features aimed to help developers build applications, administrators to protect data integrity and build fault-tolerant environments, and help you manage your data no matter how big or small the dataset. In addition to being free and open source, PostgreSQL is highly extensible. For example, you can define your own data types, build out custom functions, even write code from different programming languages without recompiling your database!

PostgreSQL tries to conform with the SQL standard where such conformance does not contradict traditional features or could lead to poor architectural decisions. Many of the features required by the SQL standard are supported, though sometimes with slightly differing syntax or function.

PostgreSQL is used as the only Relational Database Management System (RDMS) of the project.

Figure 4: PostgreSQL RDMS Logo

3.5 Spring Boot

Spring Boot is an open-source Java framework used for programming standalone, production-grade Spring-based applications with a bundle of libraries that make project startup and management easier.

Spring Boot is a convention-over-configuration extension for the Spring Java platform intended to help minimize configuration concerns while creating Spring-based applications. The application can still be adjusted for specific needs, but the initial Spring Boot project provides a preconfigured "opinionated view" of the best configuration to use with the Spring platform and selected third-party libraries. Spring Boot can also be used to build microservices, web applications, and console applications.

The main benefits of Spring Boot in relation to other Java web development frameworks include:

- Embedded Tomcat, Jetty or Undertow web application server out of the box.
- Provide opinionated 'starter' Project Object Models (POMs) for the build tool. The only build tools supported are Maven and Gradle.
- Automatic configuration of the Spring Application.
- Provide production-ready functionality such as metrics, health checks, and externalized configuration.
- No code generation is required.
- No XML configuration is required.
- Optional support for Kotlin and Apache Groovy in addition to Java.

SpringBoot is used as the development framework for the server (business logic).

Figure 5: Spring Boot Framework Logo

3.6 Angular

Angular is a TypeScript-based free and open-source single-page web application client-side framework. It is developed by Google and by a community of individuals and corporations. Its architecture is based on components and templates, each binding properties or events to each other.

Angular provides a list of features, such as:

- **Component-based architecture**: Angular uses a component-based architecture, which allows developers to build encapsulated, reusable user interface elements. Each component encapsulates its own HTML, CSS, and TypeScript, making it easier to manage and test individual pieces of an application.

- **Data binding**: Angular supports two-way data binding, which synchronizes data between the model and the view. This ensures that any changes in the view are automatically reflected in the model and vice versa.

- **Dependency injection**: Angular has a built-in dependency injection system that makes it easier to manage and inject dependencies into components and services. This promotes modularity and easier testing.

- **Directives**: Angular extends HTML with additional attributes called directives. Directives offer functionality to change the behavior or appearance of DOM elements.

- **Routing**: Angular includes a router that allows developers to define and manage application states and navigation paths, making it easier to build single-page applications with complex routing.

- **Angular CLI**: The Angular CLI (Command Line Interface) provides a set of tools for creating, building, testing, and deploying Angular applications. It enables rapid application setup and simplifies ongoing development tasks.

Angular is used as the development framework for the frontend (client).

Figure 6: Angular Framework Logo

3.7 Docker

Docker is an operating system level virtualization technology that facilitates the build process of applications, regardless of what platform and what configurations and dependencies are installed on them. This is particularly useful when developing bigger applications with a big number of engineers since there are zero dependencies-other than Docker-that are needed to build an application(s).

Figure 7: Docker Logo

3.8 Docker Compose

Docker Compose is a tool for defining and running multi-container applications. It is the key to unlocking a streamlined and efficient development and deployment experience. Compose simplifies the control of your entire application stack, making it easy to manage services, networks, and volumes in a single, comprehensible YAML configuration file. Then, with a single command, you create and start all the services from your configuration file.

Compose works in all environments, production, staging, development, testing, as well as CI workflows. It also has commands for managing the whole lifecycle of your application:

- Start, stop, and rebuild services.
- View the status of running services.
- Stream the log output of running services.

Docker and Docker Compose are used as the main build/deployment tool of the project.

Figure 8: Docker Compose Logo

3.9 Apache Maven

Apache Maven is a software project management and build tool. Based on the concept of a project object model (POM), Maven can manage a project's build, reporting and documentation from a central piece of information.

Maven's primary goal is to allow a developer to comprehend the complete state of a development effort in the shortest period. To attain this goal, Maven deals with several areas of concern:

- Making the build process easy.
- Providing a uniform build system.
- Providing quality project information.

Maven is used as the main build tool for the backend (business logic).

Figure 9: Apache Maven Logo

3.10 Bash

Bash, short for Bourne-Again SHell, is a shell program and command language supported by the Free Software Foundation and first developed for the GNU Project by Brian Fox. Designed as a 100% free software alternative for the Bourne shell, it was initially released in 1989. Its moniker is a play on words, referencing both its predecessor, the Bourne shell, and the concept of rebirth. It is ideal for scripts, configurations and automations of various build tasks such as building and deploying applications and pipelines.

Bash is used for facilitating the build/deployment process and for configuration files.

Figure 10: Bash Command Language Logo

3.11 gRPC

gRPC is a modern open-source high performance Remote Procedure Call (RPC) framework that can run in any environment. It can efficiently connect services in and across data centers with pluggable support for load balancing, tracing, health checking and authentication. It is also applicable in last mile of distributed computing to connect devices, mobile applications and browsers to backend services.

Where gRPC really outperforms other protocols (like REST) is in time critical and resource heavy operations. Such tasks can be:

- **Real-time Communication**: gRPC is ideal for real-time applications such as online gaming, live video streaming, and financial trading.

- **Microservices Intercommunication**: In a microservices architecture, where services need to communicate frequently and efficiently.

- **Big Data Processing**: gRPC is ideal for dealing with large datasets or high-volume data transfers, since it efficiently handles binary serialization using protobuf (Protocol Buffers).

- **Machine Learning**: Models, especially in low resource systems, require a particular amount of time to finish. Reducing the time of REST serialization is crucial for better performance and availability.

- **Mobile and Web Services**: Mobile applications usually make frequent and quick calls (notification systems for example). Using gRPC reduces the time of serialization of the response of the web server, improving the user experience.

gRPC is used as the communication protocol of the server and the clustering service.

Figure 11: gRPC Logo

3.12 Word2Vec

Word2Vec is a technique for natural language processing that transforms words into continuous vector representations. Developed by Google, it uses neural networks to learn word associations from large corpora of text. The resulting vectors capture semantic relationships between words, enabling tasks like word similarity and analogy solving. Word2Vec is used in the evaluation experiments.

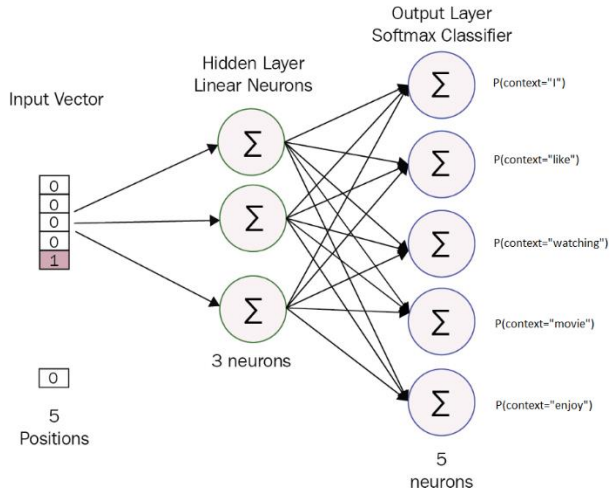


Figure 12: Word2Vec architecture illustration

3.13 Doc2Vec

Doc2Vec extends the principles of Word2Vec to entire documents. It generates fixed-length vector representations for variable-length pieces of text, such as sentences, paragraphs, or documents. This model is particularly useful for tasks like document classification, clustering, and information retrieval, as it captures the context and semantics of the text. Doc2Vec is used in the evaluation experiments.

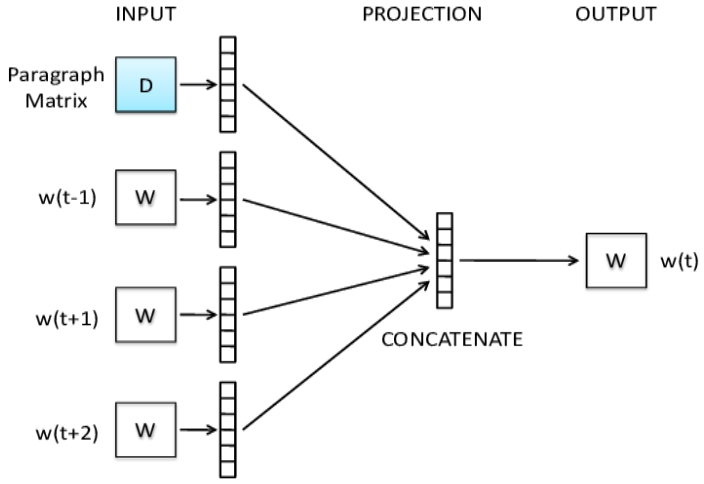


Figure 13: Doc2Vec architecture illustration

3.14 CodeBERT

CodeBERT is a transformer-based model designed for natural language and programming language understanding [3]. It is pre-trained on a large dataset of code projects and natural language comments, making it adept at tasks like code search, code summarization, and code generation. CodeBERT bridges the gap between natural language processing and software engineering, making it ideal for our use case in code semantics and structure. CodeBERT is the foundation for UniXcoder.

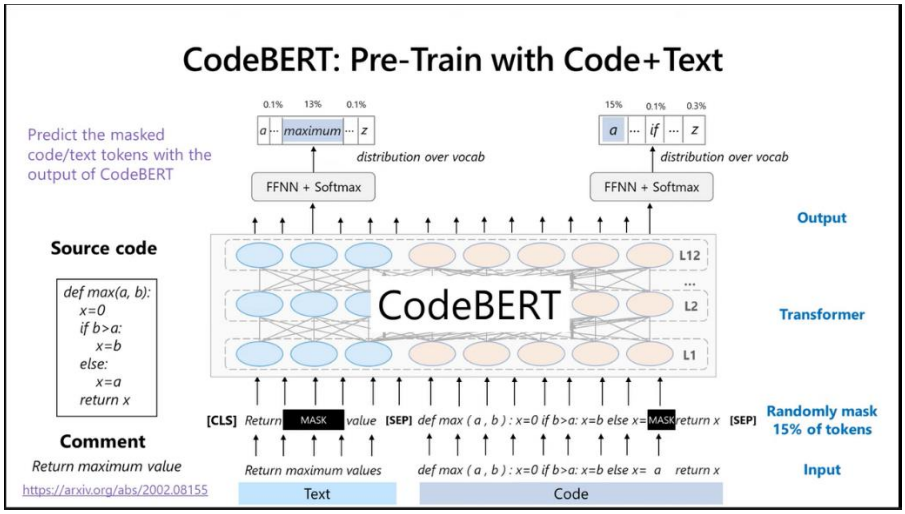


Figure 14: CodeBERT architecture illustration

3.15 UniXcoder

UniXcoder is a unified pre-trained model for cross-modal tasks involving both natural language and programming languages [4]. It is based on CodeBERT and leverages a common shared encoder-decoder architecture to handle various tasks such as code translation, code completion, and code documentation generation. UniXcoder aims to improve the efficiency and accuracy of code-related tasks by utilizing a single model for multiple purposes. UniXcoder is used in the evaluation experiments.

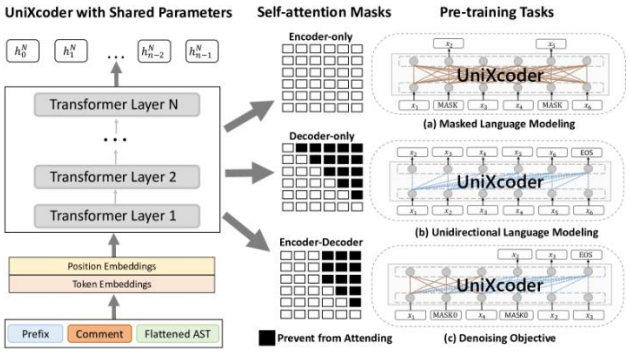


Figure 15: UniXcoder architecture illustration

3.16 PCA

Principal Component Analysis (PCA) is a dimensionality reduction technique used in machine learning and data analysis. It transforms high-dimensional data into a lower-dimensional form while preserving as much variance as possible, thus leading to the same performance with less dimensions and complexity. PCA is used in the evaluation experiments.

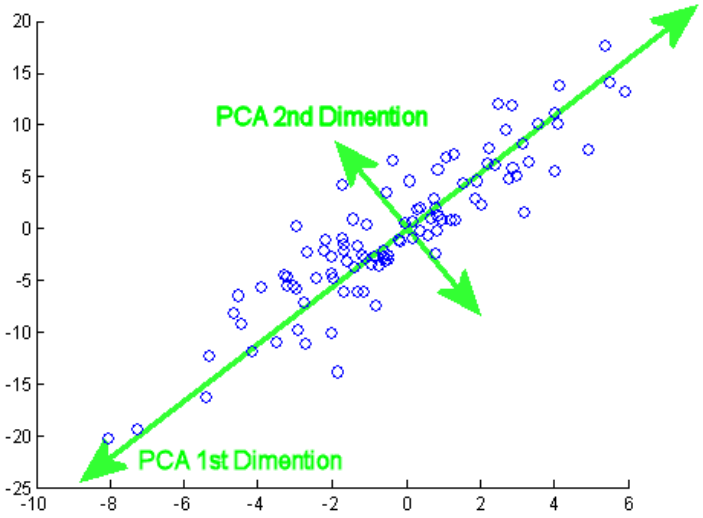


Figure 16: PCA architecture illustration

3.17 MeanShift

MeanShift is a clustering algorithm that does not require specifying the number of clusters in advance. It works by iteratively shifting data points towards the mode (highest density point) of their local neighborhood. This non-parametric approach is effective for discovering clusters of arbitrary shapes and simplicity since no parameters are required be set. MeanShift is used in the evaluation experiments.

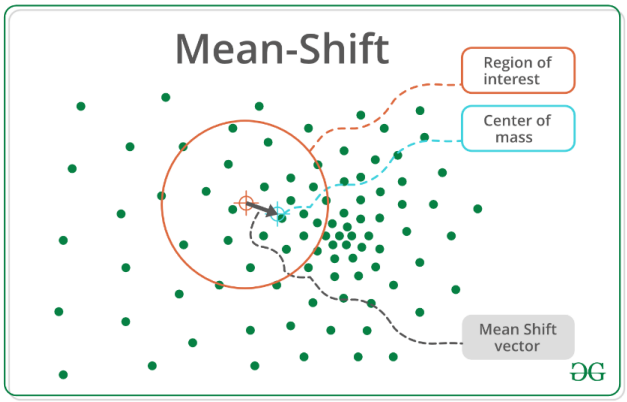


Figure 17: MeanShift architecture illustration

3.18 Affinity Propagation

Affinity Propagation is a clustering algorithm that identifies exemplars, or representative points, by passing messages between data points [6]. Unlike traditional clustering methods, it does not require the number of clusters to be specified beforehand. Affinity Propagation is known for its ability to handle large datasets and find clusters with varying sizes and densities, making it a valid candidate for performing well in large code datasets. Affinity Propagation is used in the evaluation experiments.

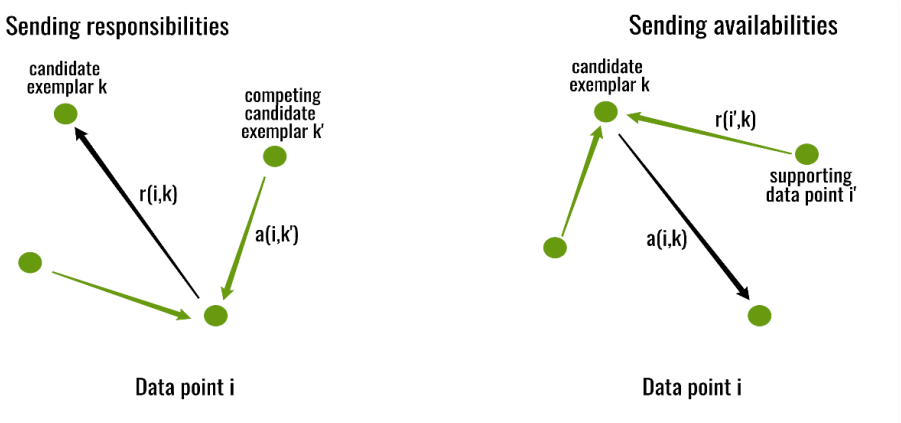


Figure 18: Affinity Propagation architecture illustration

3.19 BIRCH

Balanced Iterative Reducing and Clustering using Hierarchies is a scalable clustering algorithm designed for large datasets. It incrementally and dynamically clusters incoming data points, using a tree structure to summarize the data [8]. BIRCH is efficient in terms memory and computational resources, making it suitable for real-time applications (static code analysis in our case). BIRCH is used in the evaluation experiments.

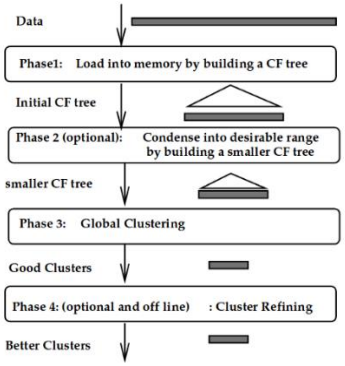


Figure 19: BIRCH architecture illustration

4 Methodology and Model Development

This section describes the methods used to create a framework for semantic and structural analysis of code files based on clustering paradigms. The proposed model consists of two basic components: an embedder, which generates meaningful representations of the files, and a clustering algorithm, which organizes these representations into coherent clusters. These components work together to make code file analysis and classification easier, regardless of framework.

The embedder is based on UniXcoder and was fine-tuned to extract context-aware embeddings from various Java programming codebases (SpringBoot, Spring and Plain Java), and the Affinity Propagation clustering algorithm was chosen to efficiently handle these embeddings. This methodology was designed with adaptability and accuracy in mind, assuring its usability across a variety of frameworks.

The remainder of this section focusses on the methodology's design and implementation. The design concepts that guided the embedder's development and selection are presented first, followed by an explanation of how the clustering method was selected and configured. Finally, an integration approach for merging these components is described, as well as the criteria for evaluating their success.

4.1 Defining Similarity

For clustering to make sense, a similarity metric must be defined beforehand. While this might be straight-forward for other kinds of data (e.g. plain text), it can be confusing for coding tasks. One might consider **UserService** and **UserController** to be similar-since they both handle user related logic-while another might consider them as dissimilar since doing otherwise does not follow the MVC (Model View Controller) paradigm.

In this version of the tool, the clustering paradigm in which the model was trained follows the MVC (Model View Controller) architecture, which states that the related program logic is divided into three interconnected elements.

These elements are:

- **Model**: internal representations of information.
- **View**: the interface that presents information to and accepts it from the user.
- **Controller**: the software linking the two.

However, the tool is designed in such a way that the addition of models with different clustering paradigms in the future is done in a convenient and easy way.

4.2 Embedder Selection

For clustering algorithms to produce insightful clusters, an appropriate embedding technique has to be considered. An embedding technique, in the context of text data (code in this case), refers to the transformation of input text data into numerical vectors, with the goal of capturing meaningful and relevant relationships within the data. These embeddings should preserve semantic, syntactic, and structural information to ensure that the clusters produced from the clustering algorithms reflect the underlying characteristics and functionalities of the code.

Embedding techniques can range from simple, interpretable methods like BoW (Bag of Words) and TF-IDF, which represent text as sparse vectors based on token frequencies or weighted importance, to more advanced, context-aware approaches such as neural network-based embeddings. For example, autoencoders compress code into latent representations, while transformer models like CodeBERT and UniXcoder utilize self-attention mechanisms to generate embeddings that capture both lexical details and high-level semantics [4]. Graph-based approaches, such as those employing Graph Neural Networks, encode structural information by representing code as ASTs (Abstract Syntax Trees) or DFGs (Data Flow Graphs) [7].

Autoencoders serve as a valuable tool for generating embeddings by learning compact latent representations of data. Through unsupervised learning, autoencoders encode input data into a lower-dimensional space and then reconstruct it, ensuring that the most critical features are preserved. In the context of source code, this approach can effectively capture syntactic and basic structural elements, enabling clustering algorithms to group code snippets with similar patterns. However, autoencoders may struggle with capturing the full semantic richness and hierarchical relationships inherent in programming languages, especially when dealing with complex or context-dependent tasks.

On the other hand, transformer-based models provide a more robust solution for embedding code. By leveraging the self-attention mechanism, transformers can model both local and global dependencies within the input data. This makes them particularly effective at representing code, where understanding context and relationships between tokens is critical. Models like CodeBERT and UniXcoder take this further by tailoring the transformer architecture to the specific needs of programming languages.

UniXcoder introduces several innovations that make it well-suited for embedding source code. It employs dual encoders to handle both natural language and programming language inputs, enabling it to bridge the gap between comments, documentation, and the code itself. Pretraining tasks such as masked language modeling and denoising are used to fine-tune the model's ability to generate embeddings that reflect variable relationships, function dependencies, and high-level semantics. Moreover, UniXcoder integrates positional encodings and token-type embeddings that are specifically designed to represent the hierarchical and structural aspects of code, such as nested loops, conditional statements, and function calls.

One notable advantage of UniXcoder is its ability to handle both lexical details and broader structural patterns. Unlike simpler techniques like BoW or TF-IDF, which focus on token-level similarities and fail to account for context, UniXcoder’s embeddings encapsulate the deeper relationships within code. This allows clustering algorithms to identify groups that not only share

similar syntax but also perform similar functionalities or operations, making the clusters more meaningful and insightful.

The choice of embedding technique directly impacts the quality and effectiveness of the clustering algorithm. While autoencoders are a practical choice for certain applications, they lack the nuanced understanding of context and relationships that transformer-based models provide. UniXcoder, with its comprehensive approach to encoding both semantic and structural information, presents a solution for embedding code in a way that aligns with the complex, hierarchical nature of programming languages. For these reasons, transformer-based models like UniXcoder are particularly well-suited for our use case, offering embeddings that ensure clustering reflects the true characteristics and functionalities of the codebase based on the MVC architectural pattern.

The eventual selection of UniXcoder will be discussed in the following sections as the experiment results are discussed.

4.3 Clustering Algorithm Selection

The clustering algorithm selection is dependent on the embedding method that will be used. Overall, the choice depends on the specific needs of the tool. Since for this version of the model we strive for a Model View Controller (MVC) clustering paradigm, it is better to have more clusters than to have fewer ones. This is a feature that the Affinity Propagation implementation in scikit-learn brings for our specific data.

The eventual selection of Affinity Propagation will be discussed in the following sections as the experiments results are discussed.

4.4 Experiment Methodology

To correctly identify and determine which embedder and clustering algorithm combination pairs relatively well for clustering code files based on the requirements, all combinations from a set of embedding techniques and clustering algorithms were performed on 3 Apache projects from GitHub. The clustering algorithms were carefully selected to be nonparametric since that guarantees consistency between different project structures. Algorithms that do not define the number of clusters were also selected since each project can have a different number of clusters. On the other hand, parametric algorithms provide more flexibility at the expense of regularity, thus making them unsuitable. The experiments evaluation code resides under `cluster/evaluate.py`.

4.4.1 Step 1

```python
# Validate arguments
is_valid, project_language, project_path, file_extensions = validate_arguments(sys.argv)
if not is_valid:
    print('Error in input, exiting...')
    exit(1)

# Check for language support
if project_language not in SUPPORTED_LANGUAGES:
    print(f'Support for language"{project_language}" does not exist. List of supported languages:\n')
    for language in SUPPORTED_LANGUAGES.keys:
        print(f'\t{language}')
    exit(1)

# Get project traverser strategy based on language
traverser_strategy, keyword_remover = SUPPORTED_LANGUAGES.get(project_language)

# Get file content data
files, file_paths = project_traverser.get_project_files(traverser_strategy,
                                                        project_path,
                                                        file_extensions)

# Get file tokens (used in certain embedding methods)
tokenized_files = []
for file_content in files:
    tokens = list(javalang.tokenizer.tokenize(file_content))
    token_values = [token.value for token in tokens]
    tokenized_files.append(token_values)
```

Figure 20: evaluate.py Step 1

In Step 1, all the preprocessing is performed by first getting the traverser strategy and the keyword remover objects. It then proceeds to traverse the project by getting the contents of the files and tokenizes their contents for Word2Vec and Doc2Vec embedders (UniXcoder does not call for tokenized input).

Since this model is fine-tuned for Java projects, the only supported traversers and keyword removers are ***JavaTraverserStrategy*** and ***JavaKeywordRemover***. Each traverser and keyword remover ought to comply to their respective interfaces:

```python
class TraverserStrategy:
    def __init__(self):
        pass

    def check_file_name(self, file_path: str) -> bool:
        pass

    def check_file_content(self, file_content: str) -> bool:
        pass
```

Figure 21: Traverser Strategy interface

```
class KeywordRemover():
    def __init__(self) -> None:
        pass

    def remove_keywords(file_tokens: List[str]) -> List[str]:
        return file_tokens
```

Figure 22: Keyword Remover interface

4.4.2 Step 2

```
# Define embedding methods
embedding_methods = [
    EmbeddingMethod("UniXcoder", UniXEmbedder(), files),
    EmbeddingMethod("Word2VecEmbedder", Word2VecEmbedder(), tokenized_files),
    EmbeddingMethod("Doc2VecEmbedder", Doc2VecEmbedder(), tokenized_files)
]

# Instantiate clustering algorithms
clustering_methods = {
    "MeanShift": MeanShift(max_iter=650, n_jobs=-1),
    "Affinity Propagation": AffinityPropagation(),
    "BIRCH": Birch()
}
```

Figure 23: evaluate.py Step 2

In Step 2, all the clustering algorithms and embeddings methods that will be tested are defined. UniXcoder uses the file content as is, while Word2Vec and Doc2Vec use the tokens of the file contents. The only tweaked hyperparameters are noticed in MeanShift, where *n_jobs=-1* tells the algorithm to use all processors, and *max_iter=650* defines the maximum number of iterations, per seed point before the clustering operation terminates (for that seed point), if has not converged yet.

4.4.3 Step 3

```python
# Try every combination to compare metrics
for embedder in embedding_methods:
    emb_name   = embedder.emb_name
    emb_method = embedder.emb_method
    emb_X      = embedder.emb_X

    emb_method.train(emb_X)
    embeddings = emb_method.get_embeddings(emb_X)

    threshold = 0.95 # Threshold for cumulative variance
    n_components = get_optimal_pca_n_components(embeddings, threshold)
    pca_embeddings = get_pca_embeddings(embeddings, n_components)

    # Here we declare a list that holds:
    # a) original embeddings
    # b) pca embeddings
    embeddings_list = [embeddings, pca_embeddings]

    # Show variance plot to find out the optimal n_components
    #show_pca_variance_plot(embeddings)

    # Show first two dimensions with PCA (with optimal n_components)
    #show_data_plot(emb_name, pca_embeddings, n_components)
```

Figure 24: evaluate.py Step 3.1

```python
    # Try each embedding
    for embedding in embeddings_list:
        print(f'{emb_name}')
        print(f'Dimension: {len(embedding[0])}')

        X_train, X_test, train_indices, test_indices = train_test_split(
            embedding, range(len(emb_X)),
            test_size=0.2, random_state=42
        )
        for cl_name in clustering_methods:
            cl_model = copy.deepcopy(clustering_methods[cl_name])
            cl_model.fit(X_train)
            Y_pred = cl_model.predict(X_test)

            num_of_clusters = len(np.unique(Y_pred))
            metrics_dict = calc_performance_metrics(X_test, Y_pred)
            print(f'\n{emb_name} -> {cl_name}')
            print(f'Number of clusters: {num_of_clusters}')
            print(f'Metrics:')
            print(metrics_dict)
            print("---------------------\n\n")
            #original_file_content = files[test_indices[i]]
```

Figure 25: evaluate.py Step 3.2

In steps 3.1 and 3.2, each combination of Principal Component Analysis, embedding method and clustering algorithm is tested. The percentage of the test data is kept at 20 percent to ensure that just enough amount of data is used in the training and in the testing process. It then calculates the performance metrics of the clustering result to get an initial glimpse of how each combination performed. The metrics that were selected were few since there do not exist many for unlabeled data (Silhouette Score, Calinski-Harabasz Index and Davies-Bouldin Index).

4.5 Experiment Results and Model Selection

The evaluation was conducted on 3 Apache Projects (Apache Tomcat, Apache Kafka and Apache Flink). The results were then exported to an excel pivot table which describes the metrics in detail. The following is a diagram depicting the Metrics Per Project (MPP):

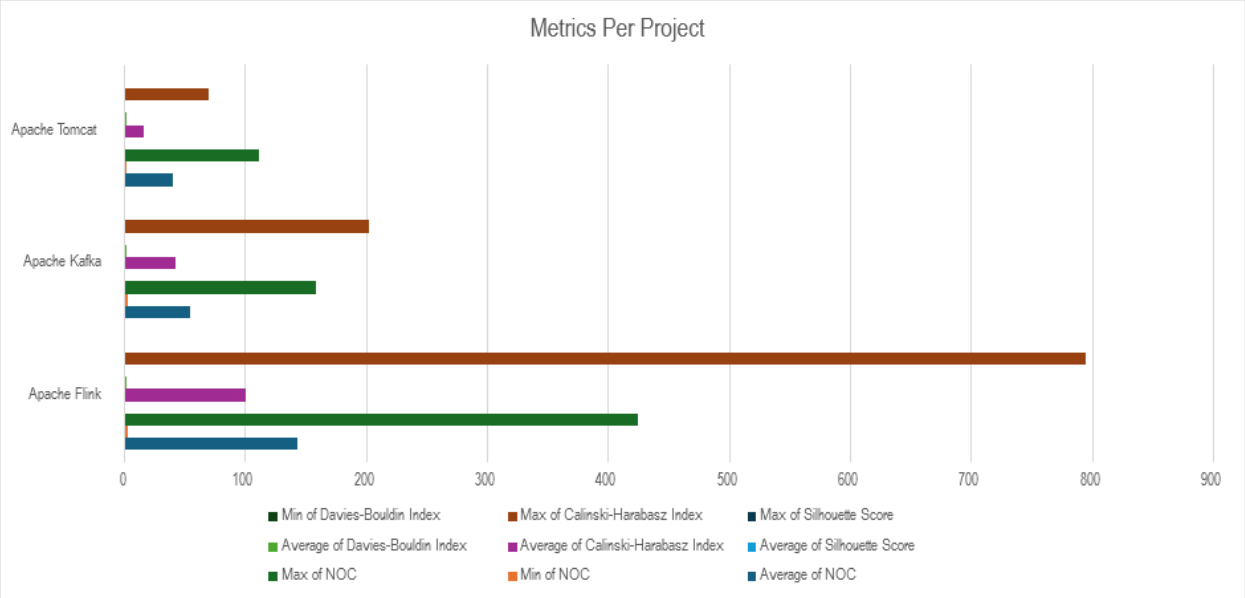


Figure 26: Metrics Per Project Diagram

As depicted in the diagram, Apache Tomcat and Apache Kafka differ in their values but not by a very large margin, whereas Apache Flink presents the most differences in respect to the other two. This could be caused by outliers or the relative size/complexity of the project. Below is the complete pivot table of the results:

Table 1: Metrics Per Project Pivot Table

Project	**Emb. Method**	**Cluster Algorithm**	**PCA**	**Average of SS**	**Average of CHI**	**Average of DBI**	**Average of NOC**
Apache Flink	**Doc2Vec**	Affinity Propagation	FALSE	-0.09441227	6.488128898	1.48513964	366
			TRUE	-0.101232988	6.816530097	1.45343186	371

		Affinity Propagation Total		-0.097822629	6.652329498	1.46928575	368.5
		BIRCH	FALSE	0.19852176	19.63887888	5.909685795	3
			TRUE	0.080160526	21.35516883	6.529097628	3
		BIRCH Total		0.139341143	20.49702385	6.219391711	3
		MeanShift	FALSE	0.10252251	6.158876352	0.704017539	243
			TRUE	0.107121637	6.303903729	0.756926382	248
		MeanShift Total		0.104822073	6.23139004	0.73047196	245.5
	Doc2Vec Total			**0.048780196**	**11.12691446**	**2.806383141**	**205.6666667**
	UniXcoder	Affinity Propagation	FALSE	-0.001684713	4.682095457	1.701286288	425
			TRUE	0.007204799	5.060760033	1.654925799	423
		Affinity Propagation Total		0.002760043	4.871427745	1.678106044	424
		BIRCH	FALSE	0.01378425	54.59621542	4.429017256	3
			TRUE	0.02597668	58.22457971	4.976780017	3
		BIRCH Total		0.019880465	56.41039756	4.702898636	3
		MeanShift	FALSE	0.174873837	2.606297583	0.947364462	5
			TRUE	0.170897466	2.628857788	0.882786411	6
		MeanShift Total		0.172885652	2.617577685	0.915075436	5.5
	UniXcoder Total			**0.065175387**	**21.299801**	**2.432026705**	**144.1666667**
	Word2Vec	Affinity Propagation	FALSE	0.04962963	32.53932167	1.761831709	245
			TRUE	0.056771577	41.97302905	1.680694761	230
		Affinity Propagation Total		0.053200603	37.25617536	1.721263235	237.5
		BIRCH	FALSE	0.181137983	735.6122234	1.53839306	3
			TRUE	0.165077957	795.1007092	1.598000652	3
		BIRCH Total		0.17310797	765.3564663	1.568196856	3
		MeanShift	FALSE	0.231730484	10.6739702	1.127347458	4
			TRUE	0.288506693	12.27662948	0.990750272	3
		MeanShift Total		0.260118588	11.47529984	1.059048865	3.5
	Word2Vec Total			**0.162142387**	**271.3626472**	**1.449502985**	**81.33333333**
Apache Flink Total				**0.092032657**	**101.2631209**	**2.229304277**	**143.7222222**
Apache Kafka	**Doc2Vec**	Affinity Propagation	FALSE	-0.12816474	5.948371582	1.466015351	123

			TRUE	-0.062368903	6.691239802	1.343003748	129
		Affinity Propagation Total		-0.095266822	6.319805692	1.40450955	126
		BIRCH	FALSE	0.17667846	7.755055468	5.092799213	3
			TRUE	0.185271719	8.475476731	5.249387016	3
		BIRCH Total		0.180975089	8.115266099	5.171093114	3
		MeanShift	FALSE	0.11537632	6.439267566	0.660340416	91
			TRUE	0.16760806	6.916389909	0.660670122	92
		MeanShift Total		0.14149219	6.677828737	0.660505269	91.5
	Doc2Vec Total			**0.075733486**	**7.03763351**	**2.412035977**	**73.5**
	UniXcoder	Affinity Propagation	FALSE	-0.0273426	3.62704779	1.712592748	151
			TRUE	-0.015390207	3.8842641	1.64176057	159
		Affinity Propagation Total		-0.021366403	3.755655945	1.677176659	155
		BIRCH	FALSE	0.043293406	23.31523477	3.454362938	3
			TRUE	0.03209391	21.34657634	4.112421732	3
		BIRCH Total		0.037693658	22.33090556	3.783392335	3
		MeanShift	FALSE	0.246332858	7.601916035	1.344056703	3
			TRUE	0.241632161	6.337609105	1.149957287	3
		MeanShift Total		0.243982509	6.96976257	1.247006995	3
	UniXcoder Total			**0.086769921**	**11.01877469**	**2.235858663**	**53.66666667**
	Word2Vec	Affinity Propagation	FALSE	0.076754596	25.21184851	1.534281564	105
			TRUE	0.078312125	31.87119061	1.535915499	98
		Affinity Propagation Total		0.07753336	28.54151956	1.535098531	101.5
		BIRCH	FALSE	0.132095862	169.0882961	1.937971563	3
			TRUE	0.200986088	202.5174776	1.507199738	3
		BIRCH Total		0.166540975	185.8028869	1.72258565	3
		MeanShift	FALSE	0.267926797	124.5460419	1.075965644	3
			TRUE	0.457496057	112.8833187	0.546598657	3
		MeanShift Total		0.362711427	118.7146803	0.81128215	3
	Word2Vec Total			**0.202261921**	**111.0196956**	**1.356322111**	**35.83333333**
Apache Kafka Total				**0.121588443**	**43.02536792**	**2.001405584**	**54.33333333**

Apache Tomcat	**Doc2Vec**	Affinity Propagation	FALSE	-0.12610832	5.844345596	1.467848356	77
			TRUE	-0.08305457	6.393307649	1.408305538	85
		Affinity Propagation Total		-0.104581445	6.118826622	1.438076947	81
		BIRCH	FALSE	0.21521103	8.57794534	4.87676506	3
			TRUE	0.247117154	11.31256959	4.117705895	3
		BIRCH Total		0.231164092	9.945257467	4.497235478	3
		MeanShift	FALSE	0.20262563	7.158276151	0.787874088	70
			TRUE	0.16729566	7.046641714	0.798700748	72
		MeanShift Total		0.184960645	7.102458932	0.793287418	71
	Doc2Vec Total			**0.103847764**	**7.722181007**	**2.242866614**	**51.66666667**
	UniXcoder	Affinity Propagation	FALSE	-0.01724078	3.133186966	1.716408063	108
			TRUE	-0.016603123	3.276562041	1.68586569	112
		Affinity Propagation Total		-0.016921952	3.204874503	1.701136877	110
		BIRCH	FALSE	0.026478136	10.96860358	3.675734032	3
			TRUE	0.030743241	11.12294351	3.627492005	3
		BIRCH Total		0.028610689	11.04577354	3.651613018	3
		MeanShift	FALSE	0.153818118	1.747883921	0.742545655	2
			TRUE	0.055730859	1.503164035	0.833740451	3
		MeanShift Total		0.104774489	1.625523978	0.788143053	2.5
	UniXcoder Total			**0.038821075**	**5.292057341**	**2.046964316**	**38.5**
	Word2Vec	Affinity Propagation	FALSE	0.033621548	13.72342954	1.377888063	91
			TRUE	0.030984303	15.8029979	1.361169464	86
		Affinity Propagation Total		0.032302926	14.76321372	1.369528764	88.5
		BIRCH	FALSE	0.16430472	67.32463144	2.010799963	3
			TRUE	0.208489223	69.6440768	1.924081458	3
		BIRCH Total		0.186396972	68.48435412	1.967440711	3
		MeanShift	FALSE	0.302266635	24.81650836	1.042775685	6
			TRUE	0.309892101	26.2518265	1.012110643	6
		MeanShift Total		0.306079368	25.53416743	1.027443164	6
	Word2Vec Total			**0.174926422**	**36.26057842**	**1.454804213**	**32.5**
Apache Tomcat Total				**0.105865087**	**16.42493892**	**1.914878381**	**40.88888889**

Grand Total				**0.106495395**	**53.57114258**	**2.048529414**	**79.64814815**

In this version of the pivot table, only the averages of the metrics were kept saving width space. The metrics that were selected as previously mentioned are Silhouette Score, Calinski-Harabasz Index, Davies-Bouldin Index and NOC (Number of Clusters). Number of clusters refers to how many clusters were created for the current combination. In general, the following is true for these metrics:

- **Silhouette Score (SS)**: Range [-1, 1]. Higher scores indicate better-defined clusters since it measures how similar an object is to its own cluster (cohesion) compared to other clusters (separation) [10].

- **Calinski-Harabasz Index (CHI)**: Range [0, +∞]. Higher scores suggest dense and well-separated clusters since it measures the ratio of the sum of between-cluster dispersion to within-cluster dispersion [11].

- **Davies-Bouldin Index (DBI)**: Range [0, +∞]. Smaller scores suggest dense and well-separated clusters since it evaluates the average similarity ratio of each cluster with its most similar (closest) cluster [12].

When analyzing clustering models for code snippet embeddings generated by UniXcoder, various approaches were considered, including the application of PCA for dimensionality reduction and different clustering algorithms. Ultimately, Affinity Propagation combined with UniXcoder embeddings, without the use of PCA, emerged as a superior choice for this initial version of the tool.

UniXcoder generates high-quality embeddings designed to capture the semantic structure of code snippets. Applying PCA, while useful for reducing dimensionality, introduces a risk of losing critical semantic information embedded in high-dimensional spaces. For our use case, where the goal is to accurately cluster code snippets based on their contextual and semantic similarity, preserving these details is important. The minimal performance improvements when Principal Component Analysis (PCA) was applied further showed its limited usefulness in this context.

Affinity Propagation has unique advantages for datasets where the number and shape of clusters are not known beforehand. Unlike algorithms such as k-means, Affinity Propagation does not require pre-specifying the number of clusters, which is particularly beneficial for code files in big projects with diverse and irregular patterns. It identifies exemplar points that serve as cluster centers [6], making it highly effective in capturing the nuanced relationships encoded by UniXcoder embeddings.

In practice, PCA can be computationally expensive and adds an extra layer in the preprocessing step. For our use case, the improvements in clustering performance achieved through Principal Component Analysis (PCA) were negligible, making the additional computational overhead unbeneficial. Directly utilizing UniXcoder embeddings retained their original structure and provided better alignment with Affinity Propagation's exemplar-based approach.

Below are the data plot diagrams of the first two dimensions of the embeddings generated by UniXcoder, Word2Vec and Doc2Vec tested on the Apache Hive project. They show the various densities, outliers and different behaviour each embedding technique has:

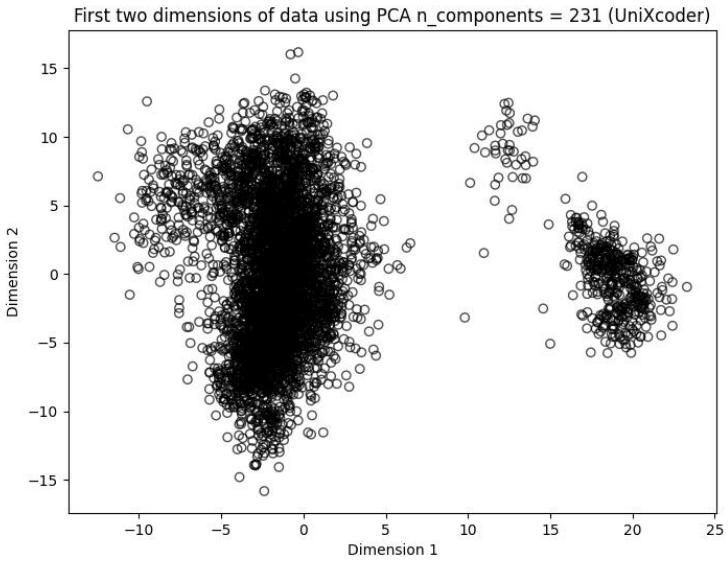


Figure 27: UniXcoder Data Plot

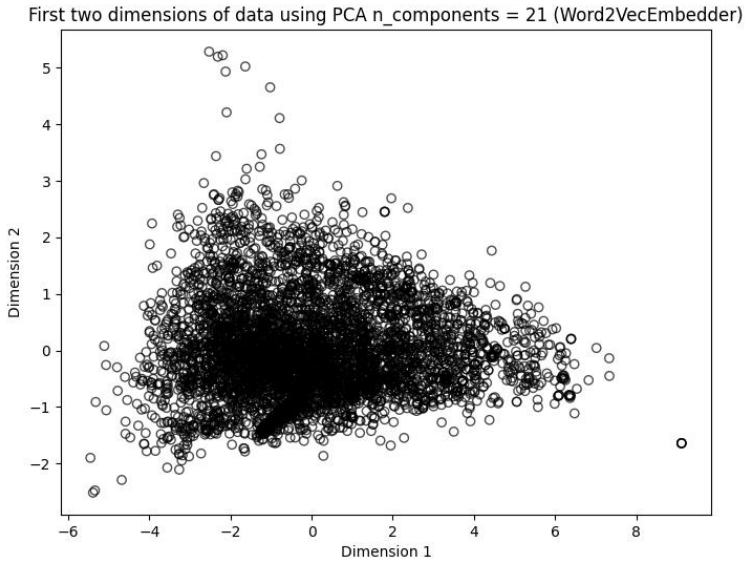


Figure 28: Word2Vec Data Plot

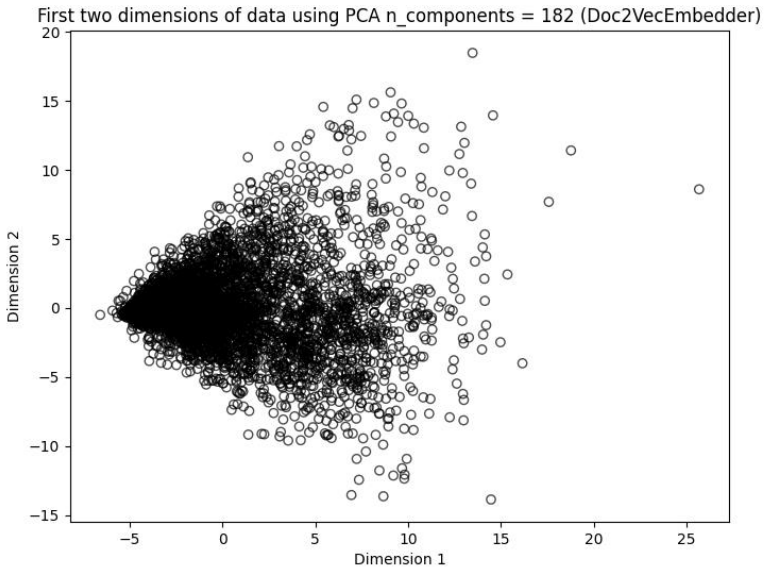


Figure 29: Doc2Vec Data Plot

In UniXcoder, we see an obvious distinction between the first two dimensions. There exist two different clusters of datapoints, one being denser than the other. This shows how UniXcoder differentiates datapoints just by the first two dimensions, while the other techniques have different shapes but follow a tendency to form dense distributions of datapoints.

The optimal PCA n components that are shown in the above diagrams were selected using the **Explained Variance Threshold** technique. This method is used to calculate how many reduced dimensions are required to confidently represent the original data dimensions [13]. The threshold of explained variance ratio was set to 95%. The following diagrams better visualize how the n components were calculated for each embedding method:

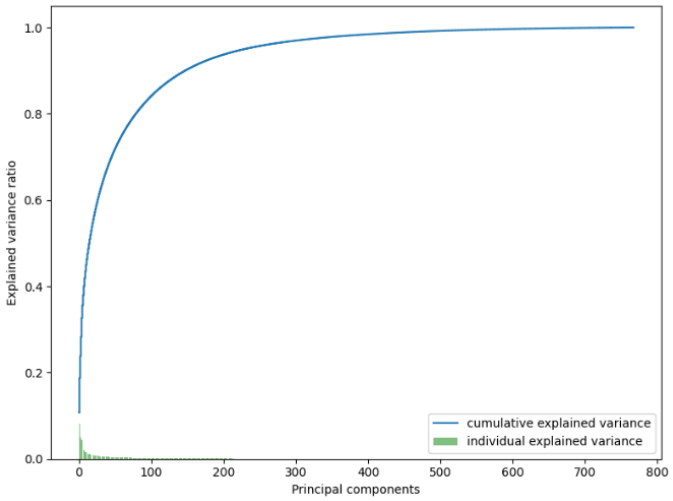


Figure 30: UniXcoder PCA Variance

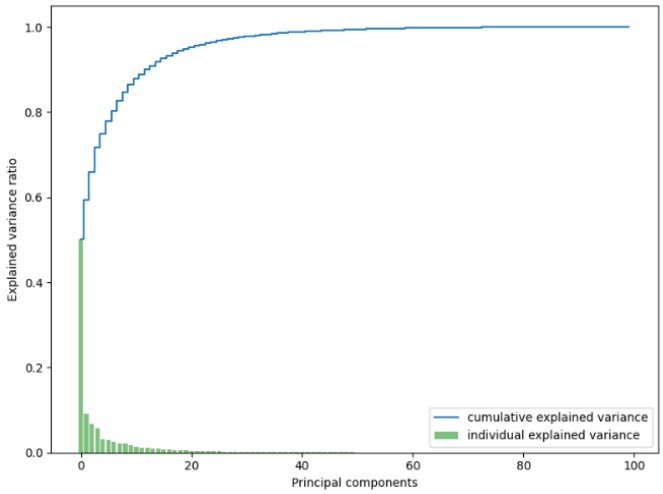


Figure 31: Word2Vec PCA Variance

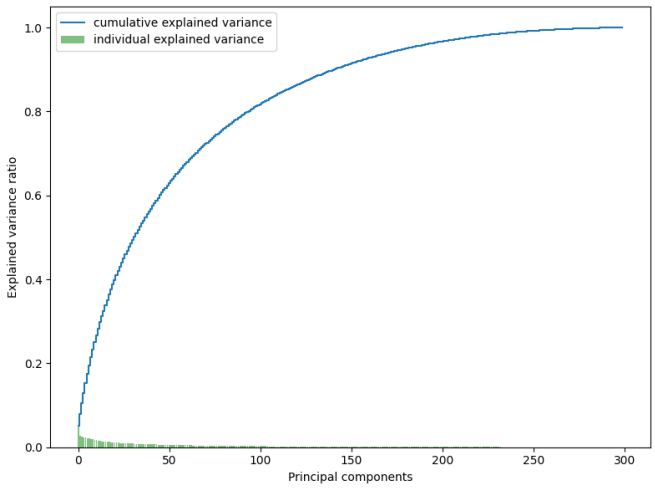


Figure 32: Doc2Vec PCA Variance

The experiment test results demonstrated that the combination of Affinity Propagation and UniXcoder embeddings without Principal Component Analysis (PCA):

- Resulted in relatively well-defined and semantically coherent clusters.
- While second on Average Silhouette Score, this combination provided better quality of clustered code files when manually analyzed.
- Reduced the risk of misrepresentation of data due to dimensionality reduction, ensuring that clusters were formed based on the full details of the embeddings.

The primary goal of the model is to group semantically similar code files in a framework-agnostic manner. Affinity Propagation's ability to identify a variable number of clusters and adapt to varying densities makes it an excellent match for our use case. By avoiding Principal Component

Analysis (PCA), the embeddings remained robust to the complexity of the original code representations.

While this combination is not the definitive solution, it is an optimal starting point for this initial version of the tool. Affinity Propagation leverages the strengths of UniXcoder embeddings without compromising their integrity through dimensionality reduction. As the project progresses, further improvements, such as tuning hyperparameters or further custom modifications on existing methods, may prove useful.

4.6 Fine-Tuning

While the combination of Affinity Propagation and UniXcoder proved to provide somewhat meaningful clusters, it also confused domain-based clustering paradigms with MVC-based ones. Clusters would often contain classes of the same domain (e.g. user) while also containing services, controllers or entities of other domains (e.g. orders). To maintain a more consistent clustering, UniXcoder was fine-tuned to support the Model-View-Controller (MVC) paradigm. To do so, a fine-tuning Python script was created.

Data used for fine-tuning tasks must have a **.acds** extension (AstroCluster Data Set). More information about their format can be found inside the project directory `cluster/finetune`. The fine-tuning was performed on 81 real world examples of code files. Most of these files were retrieved via ChatGPT prompting, and were analyzed, fixed and enriched to meet complex real-world examples. The remaining examples were retrieved from past personal projects and open-source ones. While the training data size is small, this was due to the complexity of retrieving such data (licensing issues) and time limitations. This means that the loss values were slightly unstable even if the loss value eventually got to zero after 5-6 epochs.

The **Triplet Loss** methodology was performed for training the model. In triplet loss, each datapoint consists of a pair of three elements: an anchor element, a positive element and a negative element. The anchor element can be any code file, but the positive element must be similar to the anchor and the negative element dissimilar (e.g. anchor=ClusterService, positive=EmailService, negative=UserController). This technique helps us define the dissimilar classes in a more precise manner to the model.

The fine-tuning task performs the following steps:

- Load UniXcoderForEmbedding (custom class extending UniXcoder).
- Load training data into Triplet Loss mode.
- Train the model using the AdamW optimizer [2] (Adam with Weight Decay).
- Save the fine-tuned model.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model_name = "microsoft/unixcoder-base"
model = UniXcoderForEmbedding(model_name).to(device)
tokenizer = RobertaTokenizer.from_pretrained(model_name)
triplet_loss_fn = TripletLoss(margin=1.0)

anchor_contents, positive_contents, negative_contents = load_datasets('./anchors.acds', './positives.acds', './negatives.acds')
train_dataset = TripletDataset(anchor_contents, positive_contents, negative_contents, tokenizer)
train_dataloader = DataLoader(train_dataset, batch_size=8, shuffle=True)

optimizer = AdamW(model.parameters(), lr=1e-5, weight_decay=0.01)
train_model(model, train_dataloader, optimizer, triplet_loss_fn, num_epochs=3)

# Save model
import os
# Define the directory to save the fine-tuned model and tokenizer
save_directory = './fine_tuned_unixcoder'
if not os.path.exists(save_directory):
    os.makedirs(save_directory)

# Save the model state dictionary
model_save_path = os.path.join(save_directory, 'pytorch_model.bin')
torch.save(model.state_dict(), model_save_path)

# Save the tokenizer
tokenizer.save_pretrained(save_directory)
```

Figure 33: UniXcoder Fine-tuning Script

5 Tool Development

In this section, all the decisions, technologies and technical details throughout the tool development process will be discussed. The vision and goal of this tool is to achieve several development and community-driven principles, the most important ones being convenience, performance, maintainability and open-source philosophy. The project can be found on GitHub: https://github.com/setokk/AstroCluster.git.

While this tool strives to promote good practices in software engineering, it should be noted that **maintainability** and **readability** can several times be subjective to the developer reading and analyzing the provided code base. Several factors influence the perception of a code snippet or file as "readable". These factors include:

- **Familiarity with the domain**: The name of methods or how they are used interchangeably in the code can be challenging to understand and might be deemed as "unreadable" from a reader that has not prior knowledge in the specific field (e.g. artificial intelligence). At the same time, it might be deemed "readable" from a reader with extensive experience in that domain.

- **The style of brackets and whitespaces**: Although there exist standards and guidelines that are commonly used by the respective language communities, there is still not a definitive "right" way to format code. Each project should have its own format that should be agreed on and complied with by all members.

- **The format of variables names**: A common example is the naming of list objects (e.g. *List<String> trains* instead of *List<String> trainList).* Long variables names are almost always a bad idea since the reader of the code snippet needs to read all the way of the line, making the name less memorable and the general context of it difficult to maintain. At the same time, it is also a good practice to state what exactly the variable is (in this case List).

Nevertheless, the development practices for this tool are designed to reduce the coupling of files and modules as much as possible by isolating the inevitable complexity of the problem that it is trying to solve to each service. This means that all the clustering logic and code file processing logic is handled on the Cluster service module level, all the database and client API logic is handled on the Backend service module level and all the view and presentation logic is handled on the Client service module level. This approach promotes the Singe Responsibility (SRP) principle in software development, which states that "A module should be responsible to one, and only one, actor".

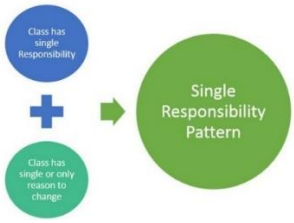


Figure 34: Diagram depicting the Single Responsibility Principle (SRP)

5.1 Architectural Decisions

5.1.1 Build Process

The build process of a project many times involves several steps. Setting up the database, inserting test data, installing various dependencies that often require specific versions to work properly, assigning environment variables, setups not working on different machines and many more. In each of these steps, there is room for error and furthermore, the principle of "convenience" is violated.

Thus, Docker Compose is used as the primary build tool of the project. As previously mentioned, Docker and Docker Compose provide several advantages over traditional build processes, the main one being consistency. Build steps will work the same way in all machines since it supports OS-level virtualization, meaning that the build steps are tested and executed via certain selected proven images.

Even through this level of automation, the user each time must explicitly provide the names of the services they want to build, copy the generated files from the docker containers to the original project directory so that the project files are updated (gRPC files generation), remove volumes or containers and use duplicate values throughout different files. Therefore a build.sh script was created since all this process can be tiring and difficult to memorize, especially in debugging.

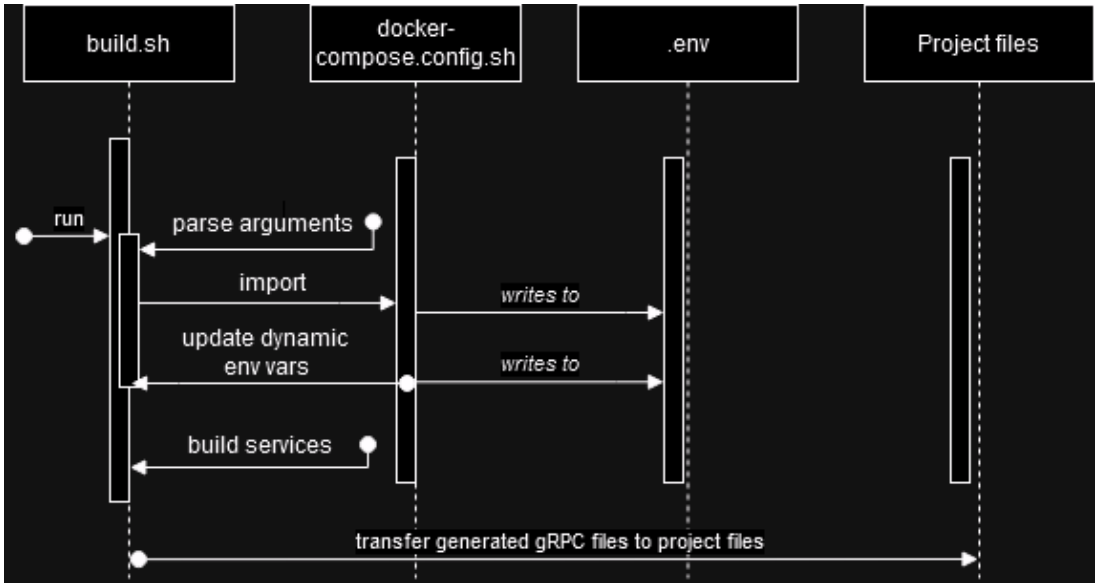
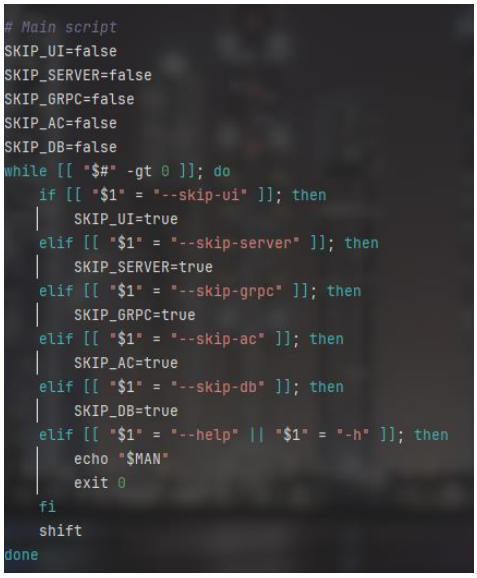


Figure 35: Diagram depicting the general project build process

The "parse arguments" phase

In this phase, the script parses the given list of arguments. Since all build arguments are in the format "—skip", their values are initially false. This means that running the build.sh script with no flags will perform a full build of the project. If the user uses the "—help" flag, the script outputs its man page explaining all the build flags and exits.

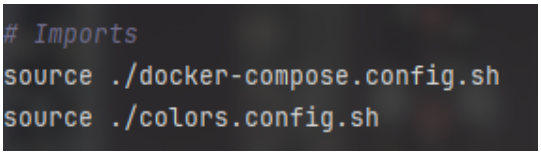


```
# Main script
SKIP_UI=false
SKIP_SERVER=false
SKIP_GRPC=false
SKIP_AC=false
SKIP_DB=false
while [[ "$#" -gt 0 ]]; do
    if [[ "$1" = "--skip-ui" ]]; then
        SKIP_UI=true
    elif [[ "$1" = "--skip-server" ]]; then
        SKIP_SERVER=true
    elif [[ "$1" = "--skip-grpc" ]]; then
        SKIP_GRPC=true
    elif [[ "$1" = "--skip-ac" ]]; then
        SKIP_AC=true
    elif [[ "$1" = "--skip-db" ]]; then
        SKIP_DB=true
    elif [[ "$1" = "--help" || "$1" = "-h" ]]; then
        echo "$MAN"
        exit 0
    fi
    shift
done
```

Figure 36: The "parse arguments" phase of build.sh

The "import" phase

In this phase, the required configs for the build process are imported. Since **grpc.config.sh** is imported inside **docker-compose.config.sh**, only **docker-compose.config.sh** and **colors.config.sh** are required for the build.sh script. More information on how **docker-compose.config.sh** configuration file affects the build process, and the side effects it has in general will be discussed in the following sub-sections.

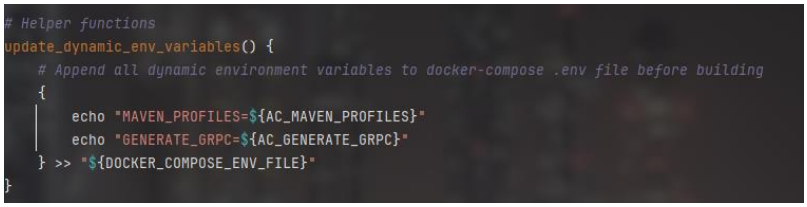


```
# Imports
source ./docker-compose.config.sh
source ./colors.config.sh
```

Figure 37: The "import" phase of build.sh

The "update dynamic env vars" phase

Besides static environment variables (container ids, paths), there also exists a need to define dynamic ones (gRPC generation flag, maven profiles). Variables of this type are appended to the previously generated **.env** file via the **update_dynamic_env_vars** function inside **build.sh**.



```
# Helper functions
update_dynamic_env_variables() {
    # Append all dynamic environment variables to docker-compose .env file before building
    {
        echo "MAVEN_PROFILES=${AC_MAVEN_PROFILES}"
        echo "GENERATE_GRPC=${AC_GENERATE_GRPC}"
    } >> "${DOCKER_COMPOSE_ENV_FILE}"
}
```

Figure 38: The "update dynamic env vars" phase of build.sh

The "build services" phase

In this phase, a validation is performed checking if no service is provided (all **–skip** flags are true), and if so, exits. Then it proceeds to build and run the services using Docker Compose's "**up**" command, followed by **-d** for detached and **--build** for forced rebuilding of the services before starting their respective containers. The services are built and started in the order in which they are added in the **SERVICES_TO_BUILD** array.

```
build_services() {
  if [ "${#SERVICES_TO_BUILD[@]}" -eq 0 ]; then
      echo -e "[${RED}BUILD ERROR${NC}]: Services to build cannot be zero. Please remove a --skip flag to continue..."
      exit 1
  fi
  echo -e "[${BLUE}BUILD INFO${NC}]: Building ${GREEN}${SERVICES_TO_BUILD[@]}${NC}"
  sudo docker-compose up -d --build "${SERVICES_TO_BUILD[@]}"
}
```

Figure 39: The "build services" phase of build.sh

The build order of services

The build order of services depends on the order in which they are defined when running the docker-compose up command (e.g. the command docker-compose up ac-db ac-server will first build and start the ac-db container, then it will build and start ac-server second). This is the default behavior of docker-compose, unless the property "depends_on" is defined in any service(s). This forces an origin service to wait for a target service to finish building, but it does not guarantee the TCP availability of the target (startup delays etc.). The way to tackle this problem will be presented in the following sections.

```
# Services to build and deploy
declare -a SERVICES_TO_BUILD
if [ $SKIP_AC = false ]; then
    SERVICES_TO_BUILD+=("clusterservice")
fi
if [ $SKIP_DB = false ]; then
    SERVICES_TO_BUILD+=("db")
    # Undeploy DB and remove docker volume with DB data
    echo -e "${BLUE}$(sudo docker stop ac-db && sudo docker rm ac-db)${NC}"
    echo -e "${BLUE}$(sudo docker volume rm pg_data_ac_cluster)${NC}"
fi
if [ $SKIP_SERVER = false ]; then
    SERVICES_TO_BUILD+=("server")
fi
if [ $SKIP_UI = false ]; then
    SERVICES_TO_BUILD+=("client")
fi
```

Figure 40: The build order of services in build.sh

5.1.2 Configuration Files

To ensure maintainability, values that are used in more than one places are defined in config.sh files. These files contain and export all variables that are used in the build.sh script in multiple places, and not only. This approach makes changing or adding variables easier and faster since doing so requires changes in less files, especially in the scenario where a single value changes (only the config.sh changes).

The config.sh files that exist in the project are **grpc.config.sh**, **docker-compose.config.sh** and **colors.config.sh**.

Configuration file **grpc.config.sh** is responsible for configuring values relevant to the gRPC generation process. It is imported in **docker-compose.config.sh**. The BE prefix stands for Backend and the CS prefix stands for Clustering Service.

```
export BE_GRPC_PROJECT_PATH="./server/src/main/java/edu/setokk/astrocluster"
export CS_GRPC_PROJECT_PATH="./cluster"
export BE_GRPC_DOCKER_PATH="/ac-server/src/main/java/edu/setokk/astrocluster/grpc"
export CS_GRPC_DOCKER_PATH="/ac-clustering-service/service"
```

Figure 41: gRPC configuration file

Configuration file **docker-compose.config.sh** is responsible for setting, exporting and writing all environment variables and arguments needed for **docker-compose.yml** to function. It is imported in **build.sh** and writes all the variables to an **.env** file recognized by Docker Compose, in the root of the project.

```
export DOCKER_COMPOSE_ENV_FILE=".env"
echo "# Auto-generated .env configuration file used by docker-compose.yml" > "${DOCKER_COMPOSE_ENV_FILE}"

# Config starts here
{
  export SERVER_CONTAINER_ID="ac-server"                  && echo "SERVER_CONTAINER_ID=${SERVER_CONTAINER_ID}"
  export CLUSTER_SERVICE_CONTAINER_ID="ac-cluster-service" && echo "CLUSTER_SERVICE_CONTAINER_ID=${CLUSTER_SERVICE_CONTAINER_ID}"
  export CLIENT_CONTAINER_ID="ac-client"                  && echo "CLIENT_CONTAINER_ID=${CLIENT_CONTAINER_ID}"
  export DB_CONTAINER_ID="ac-db"                          && echo "DB_CONTAINER_ID=${DB_CONTAINER_ID}"
  # Exported environment variables
                                                             echo "BE_GRPC_DOCKER_PATH=${BE_GRPC_DOCKER_PATH}"
} >> "${DOCKER_COMPOSE_ENV_FILE}"
echo "Generated/Overwritten .env file"
```

Figure 42: Docker Compose configuration file

Configuration file **colors.config.sh** is responsible for reducing boilerplate and duplication when handling terminal colors. It is imported in **build.sh** but can be used in any script.

```
readonly RED="\033[0;31m"
readonly GREEN='\033[0;32m'
readonly BLUE="\033[0;34m"
readonly NC="\033[0m"
```

Figure 43: Colors configuration file

5.1.3 Services

The services of this project consist of a database service, a backend service, a clustering service and a frontend service. The communication protocols that are used between the services are REST (for the bidirectional Backend and Frontend communication), default PostgreSQL protocol (for the bidirectional Backend and Database communication) and gRPC protocol (for the bidirectional Backend and Clustering Service communication).

The REST protocol is mainly used for the convenience of the client and the improved understandability of the requests and the responses. gRPC is used for critical operations and high-performance requirements. It can prove useful by removing the serialization and deserialization overhead that the REST protocol produces.

All services are defined in the docker-compose.yml file as:

- **Database Service**: The database is based on the **postgres** image. It maps and defines the ports that it uses (5432), the docker volume with the persistent DB data (pg_data_ac_cluster), and some environment variables recognized by the PostgreSQL image for setting up the default user (postgres) and the database of the application (astrocluster).

```
db:
  container_name: "${DB_CONTAINER_ID}"
  image: postgres
  ports:
    - "5432:5432"
  volumes:
    - pg_data_ac_cluster:/var/lib/postgresql/data
  environment:
    POSTGRES_USER: "postgres"
    POSTGRES_PASSWORD: "root!238Ji*"
    POSTGRES_DB: "astrocluster"
```

Figure 44: Database Service in docker-compose.yml

- **Clustering Service**: The clustering service is responsible for loading both the embedding and clustering model to perform clustering on a given set of datapoints (files). It is based on the **python:3.11-slim** image and acts as a gRPC server to communicate with the Backend service. It maps and defines the ports that it uses (50051), the docker volume with

the persistent Git projects that are cloned by the Backend (shared_cloned_projects), the docker volume with the embedding model (astrocluster_model_dir) and the build arguments used in its Dockerfile for gRPC generation (GENERATE_GRPC). Its entry point is the **init.sh** script, which will be discussed further on the **[Cluster Service]** section.

Figure 45: Clustering Service in docker-compose.yml

- **Backend Service**: The backend service is responsible for providing endpoints to the client regarding the clustering of projects, saving of analyses, history of analyses, email notifications and user management (log-in and register). In doing so, it also acts as a client to the clustering service, since it calls its performClustering method. It is based on the **openjdk:21-jdk-bullseye** for the package stage and on the **maven:3.9.9** image for its build phase. It maps and defines the ports that it uses (8080), the docker volume with the persistent Git projects (shared_cloned_projects), the environment variables used for defining the database URL (spring.datasource.url), and the build arguments used in its Dockerfile for defining the maven profiles (MAVEN_PROFILES) and the docker container directory path where the gRPC generated files are created (BE_GRPC_DOCKER_PATH).

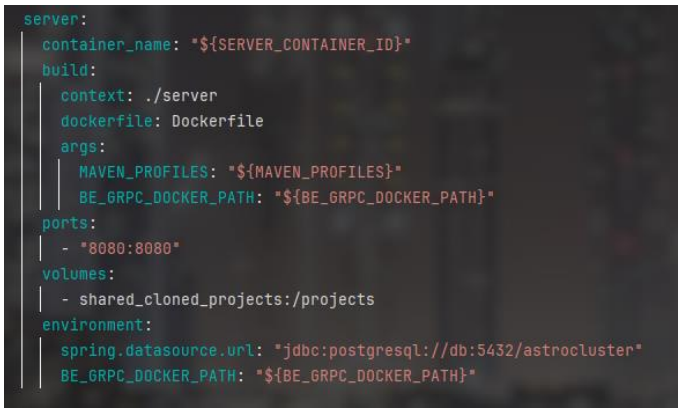


Figure 46: Backend Service in docker-compose.yml

- **Frontend Service**: The frontend service is responsible for consuming the API endpoints provided by the backend service and rendering that data to the user. It is based on the **node:18-alpine** image as its build stage, and on **nginx:alpine** as its entry point. It maps and defines the ports that it uses (80).

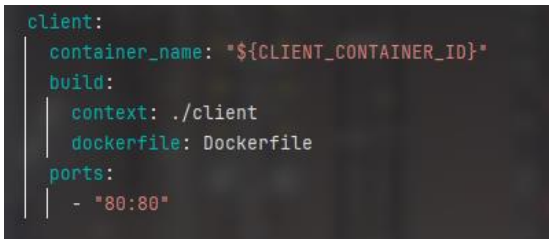


Figure 47: Frontend service in docker-compose.yml

5.1.4 Dockerfiles

Dockerfiles are a way of describing the build steps, volumes, networks and entry points of a service. Docker accepts the Dockerfiles as input and builds **images** that are run and eventually spawned as **containers**. Containers can be fully managed since Docker provides the ability to show logs, stop and start containers, delete and create volumes, copy container files to the host system both ways around, and much more. Each service (except from the Database Service) has a Dockerfile. These include:

- **Clustering Service**: Pulls the python:3.11-slim image. It then copies the dependencies separately (COPY requirements.txt) from the rest of the project files (COPY . .). This is done to utilize the Docker caching mechanism so that the project does not download the dependencies with each build (big machine learning and deep learning libraries that take time to download). Next, it installs the **gdown** tool which downloads the fine-tuned embedding model from a Google Drive link into a directory marked as a Docker volume. This is done to reduce the overall size of the project repository and avoids repository size restrictions imposed by GitHub.

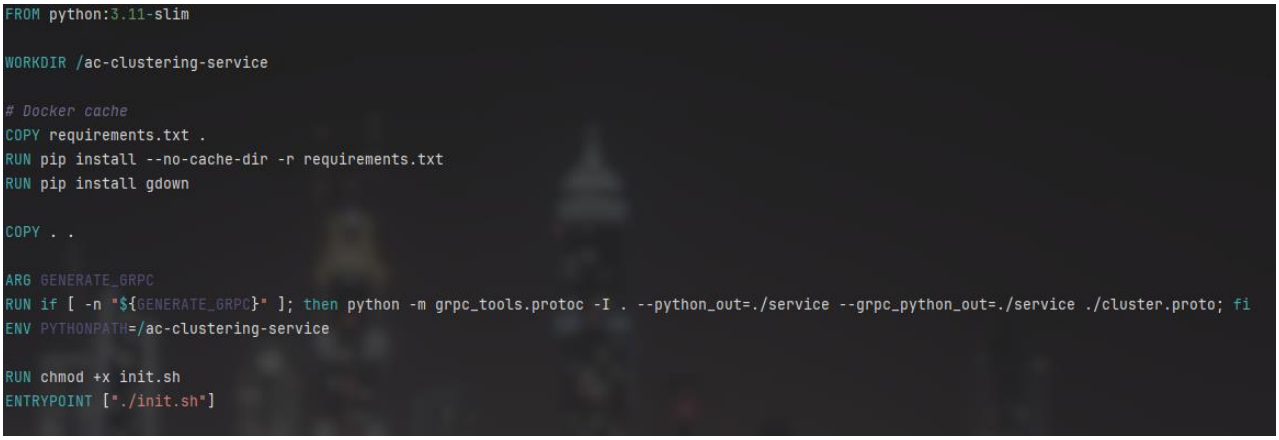


Figure 48: Clustering service Dockerfile

- **Backend Service**: Pulls maven:3.9.9 as the image for the build stage. Copies all project files (including the externalModules directory). It then performs a maven clean package, checking if there are any maven profiles provided. In the package stage, it pulls the openjdk:21-jdk-bullseye image. Bullseye was selected due to Debian's stability and security. Next, it copies the built .jar file on the root of the project along with the potentially gRPC generated files and externalModule directory.

```
# Build stage
FROM maven:3.9.9 AS build
COPY src /ac-server/src
COPY pom.xml /ac-server
COPY externalModules /ac-server/externalModules

ARG MAVEN_PROFILES
RUN if [ -z ${MAVEN_PROFILES} ]; then mvn -f /ac-server/pom.xml clean package -DskipTests; else mvn -f /ac-server/pom.xml clean package -P${MAVEN_PROFILES} -DskipTests; fi

# Package stage
FROM openjdk:21-jdk-bullseye
COPY --from=build /ac-server/target/*.jar ac-server.jar

ARG BE_GRPC_DOCKER_PATH
COPY --from=build ${BE_GRPC_DOCKER_PATH} ${BE_GRPC_DOCKER_PATH}
COPY --from=build /ac-server/externalModules /externalModules

EXPOSE 8080

COPY ./wait-for-it.sh .
RUN chmod +x ./wait-for-it.sh
ENTRYPOINT ["./wait-for-it.sh", "db:5432", "--", "java","-jar","ac-server.jar"]
```

Figure 49: Backend service Dockerfile

- **Frontend Service**: Pulls node:18-alpine as the image for the build stage. Copies all project files and installs dependencies. It then runs the build script and copies the dist directory into the nginx image which will serve as the entry point for the client on port 80.

```
FROM node:18-alpine AS build

WORKDIR /app

COPY . .

RUN npm install
RUN npm run build --prod

FROM nginx:alpine

COPY --from=build /app/dist/astro-cluster-client /usr/share/nginx/html

# Copy the modified default.conf file
COPY ./nginx/default.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

# Start Nginx
CMD ["nginx", "-g", "daemon off;"]
```

Figure 50: Frontend service Dockerfile

5.1.5 Synchronization

The synchronization of applications, especially in systems that use microservices, can prove to be a challenging task. **Startup times** must be taken into consideration and handled appropriately for every separate service. For example, the Backend service must wait for the Database service to be available via TCP to establish a connection and start executing queries on it. Doing so prematurely is guaranteed to result in errors, making the code **unreliable**.

An open-source tool that is designed for startup time synchronization and is used in the build process of this project is **wait-for-it.sh**. As the name implies, it attempts to perform a TCP connection to a host. This can be used as a blocking process before running the actual service entry point (running the jar in the case of the Backend Service).

Furthermore, **timeouts** must be considered since their absence might result in a deadlock situation. Service A waits for Service B, but a fatal error occurs on Service B during startup. Service A ought to have a timeout (e.g. 15 seconds) so that it is not stuck waiting for B indefinitely. Service A then should inform the consumers of its API that Service B is unavailable until further notice.

5.1.6 Project Tree Structure

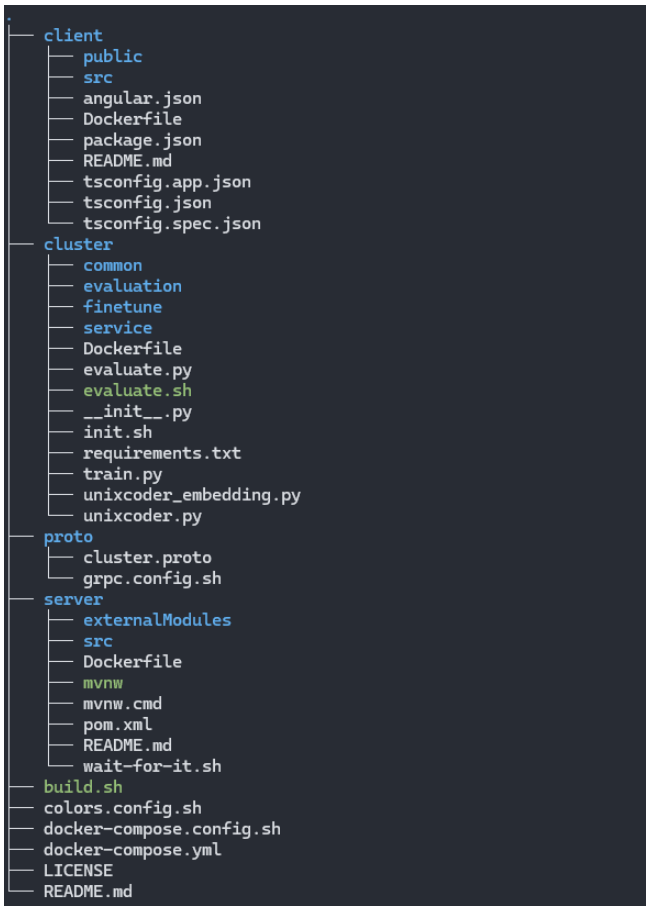


Figure 51: Project Tree Structure L2 Overview

5.2 Backend Development

In this section, the items that will be discussed and analyzed are the SpringBoot API reference, the PostgreSQL database schema specifics, the internal working mechanism of the cluster service and the communication between these modules.

5.2.1 API Reference

Before proceeding with the API reference, the following symbols must be explained:

- (**!**): Describes the **MANDATORY** presence of a field.
- (**?**): Describes the **OPTIONAL** presence of a field.
- (**??[id]**): Describes the **CONDITIONAL OPTIONAL** presence of a field (e.g. `foo??[0], bar??[1], foobar??[2], …, foobarfoobar??[N]`). The id is unique for each **CONDITIONAL OPTIONAL** entry in the scope of a Request Body definition.
- (**{}**): Notes that the field is an URL parameter (path variable).
- (***DTO***): The use of the turquoise color and the combination of both bold and italics in types indicates that the type is a business domain one (custom type). The details of these types will be explained in later sections.

The list of endpoints provided are:

```
-----------------------------------------------------------------
[POST] -> /api/cluster/perform-clustering

Description:

Calls the cluster service and performs clustering on the given
Git project (either synchronously or asynchronously). It also
persists the analysis results in a database.

-----------------------------------------------------------------
Request body: {

  gitUrl! (string): the HTTPS Git URL of the public project,

  lang! (string=SupportedLanguages[id]): the language of the
project,

  extensions? (string[]): the custom file extensions for
the cluster service to look for when traversing the Git project,

  clusteringParadigm! (string=SupportedClusteringParadigms[id]):
the clustering paradigm to be used (essentially the embedder
model),

  isAsync! (boolean): flag that indicates whether the cluster
operation should be performed asynchronously,
```

email??[0] (string): the email to which the link containing the analysis will be sent to.

}

Response body: {

ack! (short): flag indicating the successful acknowledgement of the request in the case of an async request (=0 when isAsync=false),

isAsync! (boolean): flag that indicates whether the request was made asynchronously or not,

analysisId! (long): the id of the persisted analysis (=null when isAsync=true since the request is made in the background rather than responding with the id immediately.

}

Errors body: {

errors! (string[]): a list of possible errors with prefix "[PerformClusteringRequest]:".

}

Conditional Optionals: {

email??[0]: the email is required only when no user is signed-in (no Authorization Bearer token provided in request headers).

}

[GET] -> /api/analysis/{analysisId}

Description:

Retrieves an analysis with the specified id.

Request body: {

{analysisId!} (long): the id of the analysis.

}

Response body: {

analysisData! (***AnalysisDto***): the main data of the analysis,

percentagesPerCluster! (***PercentagePerClusterDto***): the percentages of the cluster labels inside the project.

}

Errors body: {

errors! (string[]): a list of possible errors.

}

Conditional Optionals: N/A

[GET] -> /api/analysis/latest-analyses

Description:

If signed-in, retrieves the latest analyses of the user.
Otherwise, returns the latest public analyses.

Request body: N/A

Response body: {

latestAnalyses! (***AnalysisDto[]***): the main data of the analyses (without retrieving ***AnalysisDto.clusterResults***).

}

Errors body: {

errors! (string[]): a list of possible errors.

}

Conditional Optionals: N/A

[POST] -> /api/analysis/interest/csv

Description:

Calculates the technical debt interest of the project based on certain parameters and returns a CSV file containing that data.

Request body: {

analysisId! (long): the id of the analysis,

similarFilesCriteria! (string=***SimilarFilesCriteria[id]***): the file similarity metric the interest will be calculated with,

isDescriptive! (boolean): flag indicating whether to generate interest with descriptive output (explaining the calculation process),

avgPerGenerationLOC? (double): the average lines of code added in the project per generation (default value=40),

perHourLOC? (double): the lines of code added in the project per hour (default value=25),

perHourSalary? (double): the employee salary for the project per hour (default value=45).

}

Response body: {

(byte[]): byte array containing the downloadable file.

}

Errors body: {

errors! (string[]): a list of possible errors with prefix "[InterestPdfAnalysisRequest]:".

}

Conditional Optionals: N/A

[POST] -> /api/analysis/interest/pdf

Description:

Calculates the technical debt interest of the project based on certain parameters and returns a PDF file containing that data.

Request body: {

analysisId! (long): the id of the analysis,

similarFilesCriteria! (string=***SimilarFilesCriteria[id]***): the file similarity metric the interest will be calculated with,

isDescriptive! (boolean): flag indicating whether to generate interest with descriptive output explaining the calculation process or not,

avgPerGenerationLOC? (double): the average lines of code added in the project per generation (default value=40),

perHourLOC? (double): the lines of code added in the project per hour (default value=25),

perHourSalary? (double): the employee salary for the project per hour (default value=45).

}

Response body: {

(byte[]): byte array containing the downloadable file.

}

Errors body: {

errors! (string[]): a list of possible errors with prefix "[InterestPdfAnalysisRequest]:".

```
}
-----------------------------------------------------------------
Conditional Optionals: N/A
-----------------------------------------------------------------
```

```
-----------------------------------------------------------------
[POST] -> /api/users/register

Description:

Registers a new user.

-----------------------------------------------------------------
Request body: {

  email! (string): the email for the user,

  username! (string): the username for the user,

  password! (string): the password for the user.

}
-----------------------------------------------------------------
Response body: {

  (string): the JWT token in plain text.

}
-----------------------------------------------------------------
Errors body: {

  errors! (string[]): a list of possible errors with prefix
"[RegisterRequest]:".

}
-----------------------------------------------------------------
Conditional Optionals: N/A
```

```
-----------------------------------------------------------------
[POST] -> /api/users/log-in

Description:

Attempts to log-in a user.
```

```
-----------------------------------------------------------------
Request body: {

  username! (string): the username of the user,

  password! (string): the password of the user.

}
-----------------------------------------------------------------
Response body: {

  (string): the JWT token in plain text.

}
-----------------------------------------------------------------
Errors body: {

  errors! (string[]): a list of possible errors with prefix
"[LoginRequest]:".

}
-----------------------------------------------------------------
Conditional Optionals: N/A
-----------------------------------------------------------------
```

5.2.2 Domain Types

This section contains all the domain types that are common throughout all services, ensuring a unified message passing standard. The following are the complete type definitions of DTOs in AstroCluster:

```
-----------------------------------------------------------------
Name: AnalysisDto
-----------------------------------------------------------------
Domain: Analysis
-----------------------------------------------------------------
Fields: {

    id (long): id of this analysis,

    gitProjectName (string): git name of the project,

    gitUrl (string): git URL of the project,

    projectUUID (string): generated UUID of the project,

    projectLang (string): language of project
```

```
    createdDate (date, "yyyy-MM-dd'T'HH:mm:ss.SSS"): date the
analysis was created,

    clusterResults (ClusterResultDto): the results of the
analysis.

}
-----------------------------------------------------------------
```

```
-----------------------------------------------------------------
Name: ClusterResultDto
-----------------------------------------------------------------
Domain: Analysis
-----------------------------------------------------------------
Fields: {

    id (long): id of this cluster result,

    filename (string): filename of this cluster result,

    filepath (string): filepath of this cluster result,

    clusterLabel (int): cluster label assigned to this cluster
result.

}
-----------------------------------------------------------------
```

```
-----------------------------------------------------------------
Name: PercentagePerClusterDto
-----------------------------------------------------------------
Domain: Analysis
-----------------------------------------------------------------
Fields: {

    clusterLabel (int): a cluster label,

    percentageInProject (double): percentage in project of this
cluster label.

}
-----------------------------------------------------------------
```

```
-----------------------------------------------------------------
Name: UserDto
-----------------------------------------------------------------
Domain: User
-----------------------------------------------------------------
Fields: {

    id (long): the user id,

    username (string): the username of the user,

    email (string): the email of the user.

}
-----------------------------------------------------------------
```

5.2.3 Server

The server was developed using SpringBoot for its ease of use, scalability and most importantly its embedded Apache Tomcat web server. The main protocol used for communication from and to clients is REST. This does not include the communication between the server and the cluster service, since they use gRPC for more efficient and performant data transfer (eliminating JSON parsing time).

The server package hierarchy follows the MVC (Model-View Controller) pattern:

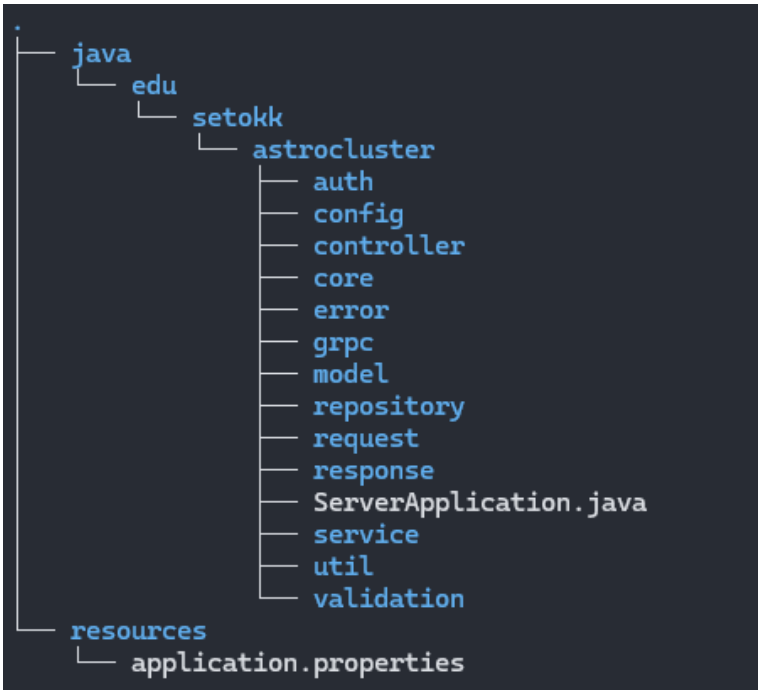


Figure 52: The server package hierarchy tree

Auth package
The auth package provides all utilities and filters that are required for the authorization process. JWTs (JSON Web Tokens) are used since they provide simplicity on the server side. The client stores the token generated by the server inside its local storage and can then use it in all the requests it makes.

Config package
The config package is responsible for configuring different parts of the application (e.g. security, thread pool management and gRPC calls). All classes that are contained within this package are annotated with @Configuration.

Controller package
The controller package consists of all controllers used in the application. They serve as the API endpoints that are exposed to the client.

Core package
The core package holds all the classes that are relevant to the business logic of the application but cannot be categorized as any of the other packages.

Error package
The error package holds all classes that are used for error and exception handling.

Grpc package
The grpc package holds all auto-generated classes from the gRPC compiler.

Model package
The model package consists of all entity classes (Java Persistence Objects) and all DTOs (Data Transfer Objects).

Repository package
The repository package consists of all classes that manipulate and query the database directly.

Request package
The request package holds all request bodies that are mostly used in controllers (POST, PUT requests).

Response package
The response package holds all response bodies that are mostly used in controllers (regardless of origin HTTP method).

Service package
The service package consists of all service classes. They are mostly used directly in controllers but can also be used interchangeably within other services.

Util package
The util package holds all utility classes that are used in different places across the application (e.g. CSV and StringUtils).

Validation package
The validation package consists of all classes that handle the validation process of the requests.

5.2.4 Cluster Service
The cluster service is responsible for loading the fine-tuned embedder model, the clustering algorithm and exposing a gRPC endpoint for the server to use. The following code snippet is the main body of the Python gRPC server code:

```python
def serve() -> None:
    logging.info("sup")
    # Load the fine-tuned model and tokenizer
    model_name = "microsoft/unixcoder-base"
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Load embedder model and tokenizer
    tokenizer = RobertaTokenizer.from_pretrained('./finetune/fine_tuned_unixcoder')
    model = UniXcoderForEmbedding(model_name)
    model.load_state_dict(torch.load('./finetune/fine_tuned_unixcoder/pytorch_model.bin', map_location=device))
    model.to(device)
    model.eval()

    # gRPC server
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    cluster_pb2_grpc.add_ClusterServiceServicer_to_server(ClusterServiceServicer(model, tokenizer, device), server)
    server.add_insecure_port('[::]:50051')
    server.start()
    logging.info("Server started on port 50051")
    server.wait_for_termination()

if __name__ == '__main__':
    serve()
```

Figure 53: Python gRPC Server code

The server first loads the fine-tuned embedder model and sets it to eval mode. This disables specific behaviors like dropout and ensures that batch normalization layers use the learned running statistics instead of batch-specific stats. Next it configures the gRPC server by providing the implementation of the ClusterServiceServicer and adding the predefined gRPC port 50051 as insecure.

5.3 Frontend Development

For the UI development, the Angular framework was used. The setup included a simple component-based architecture with CSS as the styling technology. The use of libraries was limited, apart from D3 and ngx-charts for the construction of custom graphs and charts for better analysis data visualization. In this section, the emphasis will be on the user manual rather than the code since the code is straightforward Angular architecture/concepts.

5.3.1 User Manual

The user is redirected to the home page. From there, they can interact with the top navigation bar to perform various actions (About page, Analysis Form, History and Login). They can also navigate to the Analysis Form page by clicking on the "Get started" label.

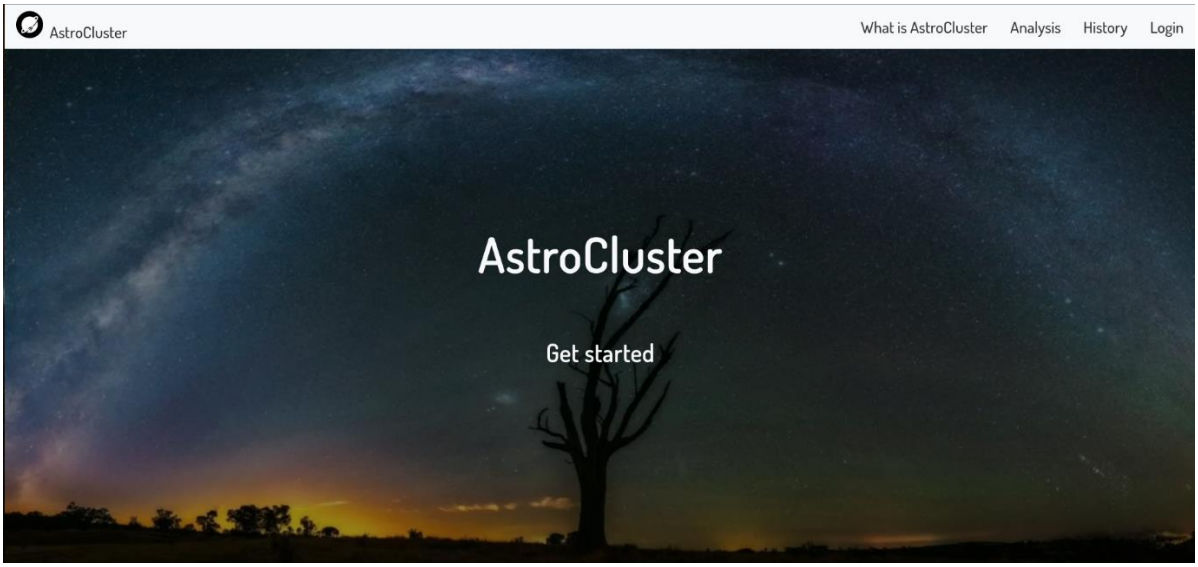


Figure 54: Home UI

In the Analysis Form, the user can complete the provided fields to perform an analysis:
Git URL: The Git URL of the project (the project must be publicly visible).

Email (optional): The email address in which the analysis completion email will be sent (optional if user is signed-in).

Asynchronous analysis flag: Indicates that the analysis will be performed asynchronously. This means that the user does not need to wait in the UI for the analysis to finish. Useful for scenarios of big projects.

Main language of project: The language of the project files the user wants to analyze.

Clustering paradigm: The logic the clustering is going to be based on.

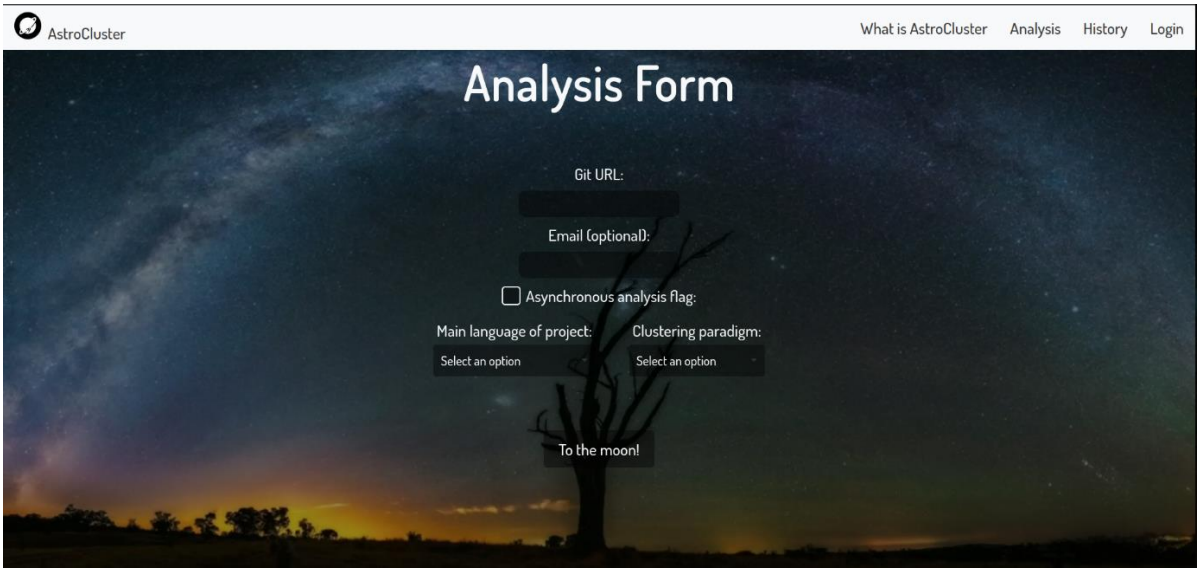


Figure 55: Analysis Form UI

After the analysis is completed, an email is going to be sent to the provided address (whether the user performed the analysis asynchronously or not). In the email, the user is able to click on the link of the analysis results.

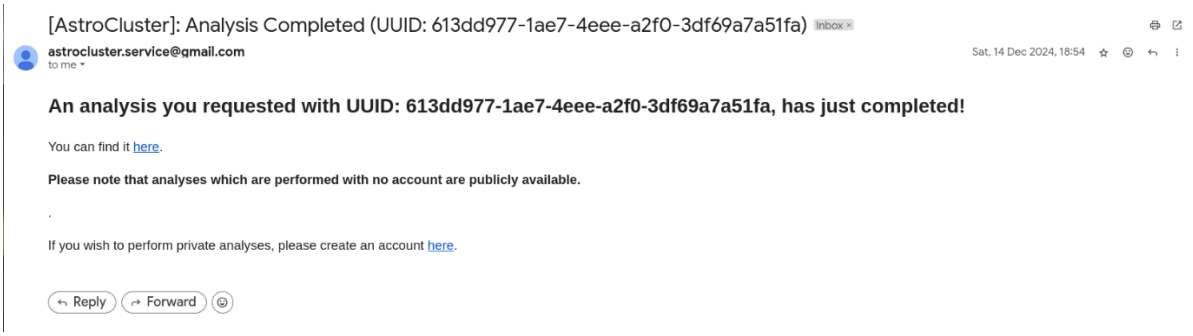


Figure 56: Analysis Completion Email UI

On the Analysis Results page, the user can view the Cluster File Results section. In this section, a graph of the project file clusters is displayed and can be better viewed while in fullscreen by clicking on the fullscreen button. The user can also select whether they want to view all clusters or only a specific one while also being able to view all the project details (Name, Git URL, Project Language and Created Date). The user can change to the Percentages per Cluster section by clicking on the “Cluster File Results” label.

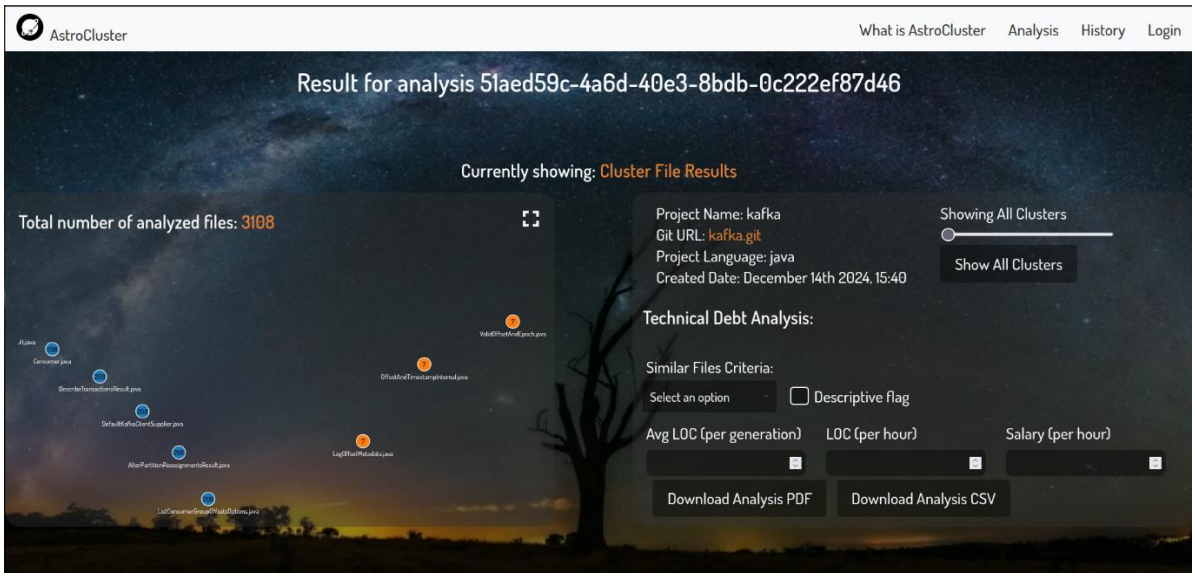


Figure 57: Cluster File Results UI

On the Percentages per Cluster section, the user can view the percentages each cluster label has in the whole project as well as how many files each cluster consists of. Just like the Cluster File Results section, the user can download a PDF or a CSV file containing the technical debt calculation analysis. This technical debt analysis can be either calculated the normal way (SIZE1 and SIZE2 metrics to determine similarity) or by using the clusters to determine similarity.

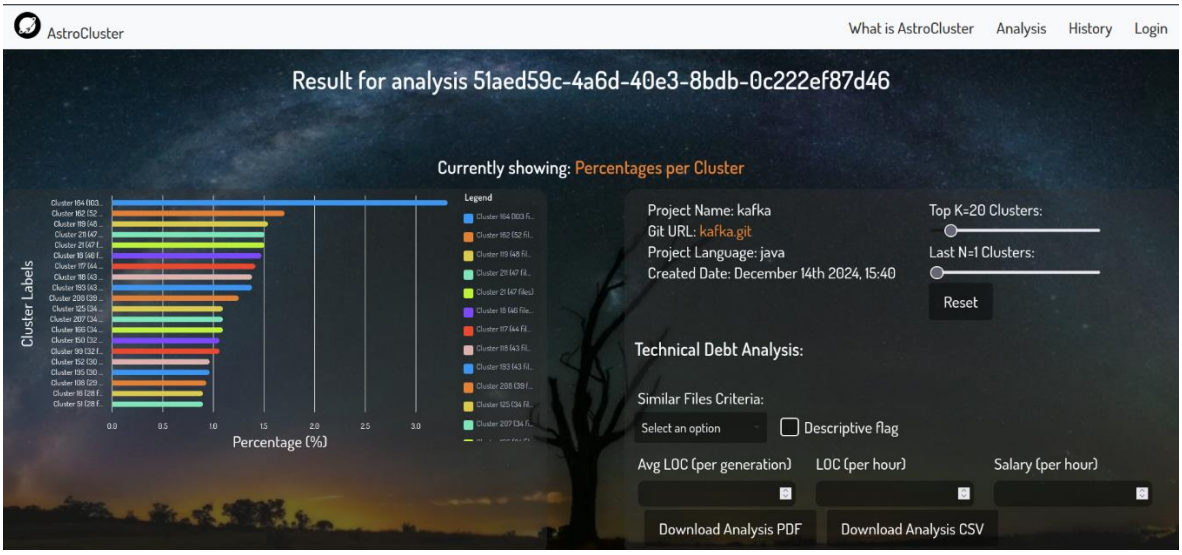


Figure 58: Percentages per Cluster UI

By navigating on “History”, the history of all the latest performed analyses can be viewed. In this case, no user is signed in, so the history is public.

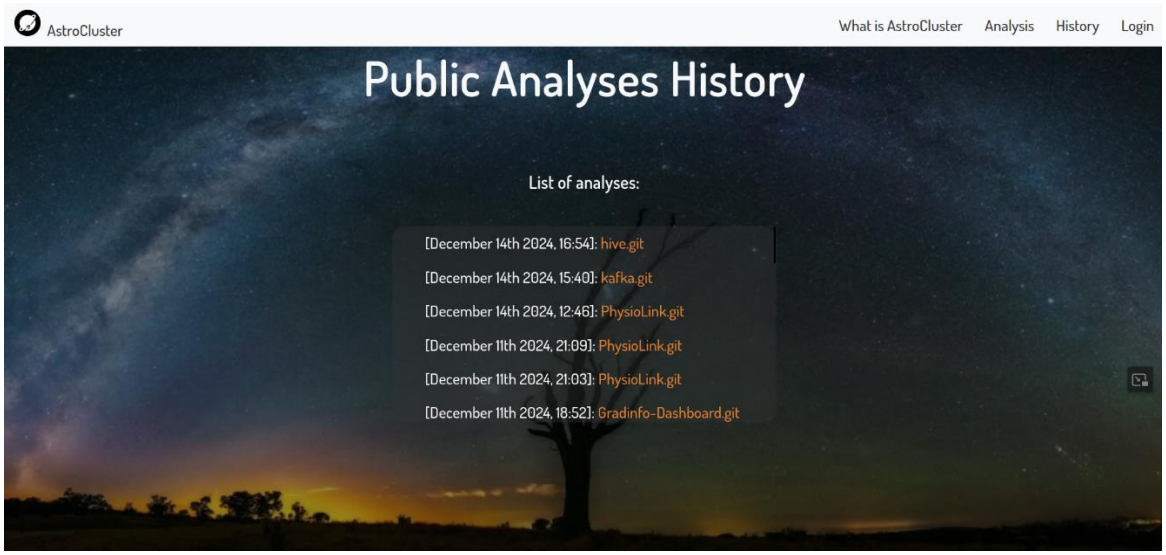


Figure 59: Public Analyses History UI

In the case where the user is signed it, they can view their personal history of analyses only.

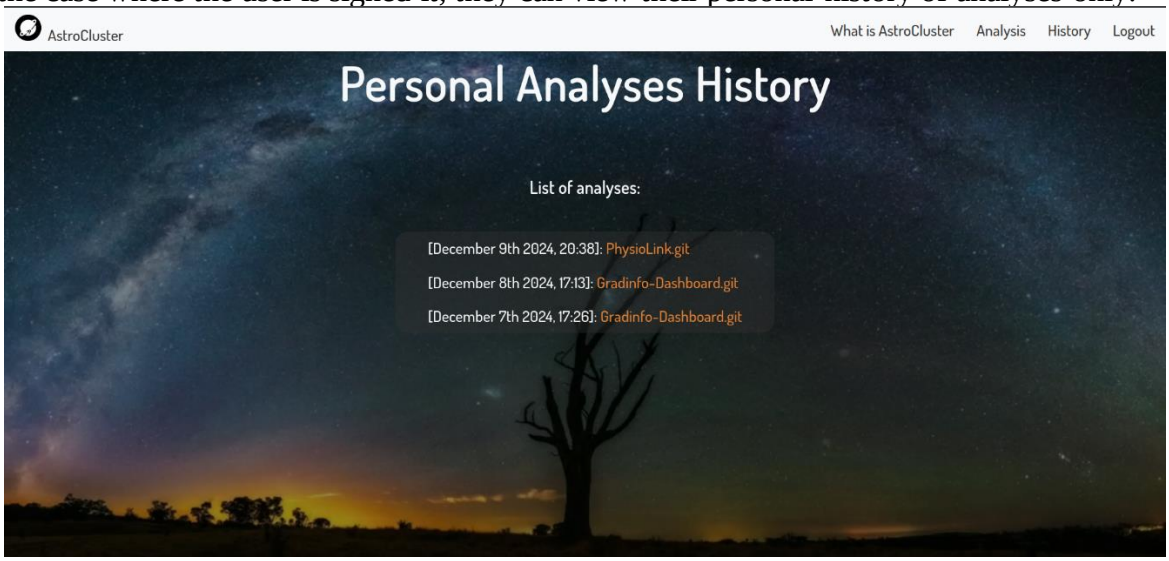


Figure 60: Personal Analyses History UI

6 Utilization for Technical Debt Calculation

Besides manual inspection for refactoring and modularization purposes, the analysis results can be used to extract technical debt details. This can done through the "Technical Debt Analysis" subsection on UI pages **"Figure 57: Cluster File Results UI"** and **"Figure 58: Percentages per Cluster UI"**.

The results can be performed either using the normal way (similarity of files is based on SIZE1 and SIZE2 metrics) or using the cluster labels to determine similar files. The metrics that are utilized for the calculation process are CBO, LCOM, WMC and DIT. The number of similar classes are equal to the size of the smallest cluster that also contains more than 3 files. The files that are not part of a cluster with size >= 3 will be skipped.

In the case of calculating the similar files using the cluster criteria, the files are not selected randomly from their respective clusters. The ones that are closer to the original file's metrics will be selected, as shown here:

```
.mapToObj(i -> {
    int otherIndex = parameters.projectFilesToMetricsCsvIndexBridge().get(projectFiles.get(i).getFilepath());
    double similarity = 0.0;
    for (var entry : metricsCsvContent.entrySet()) {
        String header = entry.getKey();
        if (header.equalsIgnoreCase( anotherString: "Name")) continue;

        double currMetricValue = Double.parseDouble(entry.getValue().get(currIndex));
        double otherMetricValue = Double.parseDouble(entry.getValue().get(otherIndex));
        if (currMetricValue == 0 && otherMetricValue == 0) {
            similarity += 1.0; // Perfect match if both are zero
        } else if (currMetricValue != 0) {
            similarity += 1 - Math.abs((otherMetricValue - currMetricValue) / currMetricValue);
        }
    }
    double avgSimilarity = (similarity / (metricsCsvContent.size() - 1)) * 100;
    return new Similarity(otherIndex, avgSimilarity);
}) Stream<Similarity>
```

Figure 61: Cluster Similar Files Logic

The following figure is an example of a generated PDF with the "Descriptive flag" unchecked:

Cluster Interest Results for analysis uuid 4d444333-e99b-46c1-8fb9-64ebed2799d3

Name	Interest in Average LOC	Interest in Hours	Interest in Dollars
/app/src/main/java/com/mobile/physiolink/util/image/ProfileImageProvider.java	171.11111	6.84444	308.00000

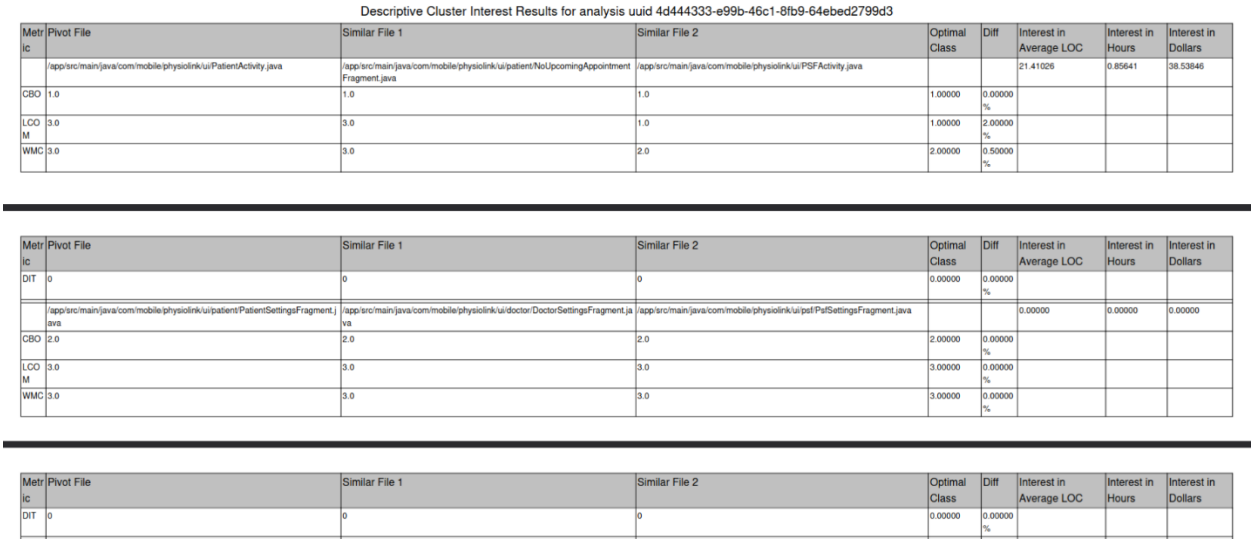
Name	Interest in Average LOC	Interest in Hours	Interest in Dollars
/app/src/main/java/com/mobile/physiolink/ui/patient/PatientSettingsFragment.java	0.00000	0.00000	0.00000

Name	Interest in Average LOC	Interest in Hours	Interest in Dollars
/app/src/main/java/com/mobile/physiolink/ui/psl/PslChangePasswordFragment.java	0.45045	0.01802	0.81081

Name	Interest in Average LOC	Interest in Hours	Interest in Dollars
/app/src/main/java/com/mobile/physiolink/LoginActivity.java	5.71429	0.22857	10.28571

Figure 62: Interest PDF Simple

The following figure is an example of a generated PDF with the "Descriptive flag" checked. It presents in extensive detail how each interest value was calculated by showing the metrics used, the optimal class and the difference from the optimal class per metric.



Descriptive Cluster Interest Results for analysis uuid 4d444333-e99b-46c1-8fb9-64ebed2799d3

Metric	Pivot File	Similar File 1	Similar File 2	Optimal Class	Diff	Interest in Average LOC	Interest in Hours	Interest in Dollars
	/app/src/main/java/com/mobile/physiolink/ui/PatientActivity.java	/app/src/main/java/com/mobile/physiolink/ui/patient/NoUpcomingAppointmentFragment.java	/app/src/main/java/com/mobile/physiolink/ui/PSFActivity.java			21.41026	0.85641	38.53846
CBO	1.0	1.0	1.0	1.00000	0.00000 %			
LCOM	3.0	3.0	1.0	1.00000	2.00000 %			
WMC	3.0	3.0	2.0	2.00000	0.50000 %			

Metric	Pivot File	Similar File 1	Similar File 2	Optimal Class	Diff	Interest in Average LOC	Interest in Hours	Interest in Dollars
DIT	0	0	0	0.00000	0.00000 %			
	/app/src/main/java/com/mobile/physiolink/ui/patient/PatientSettingsFragment.java	/app/src/main/java/com/mobile/physiolink/ui/doctor/DoctorSettingsFragment.java	/app/src/main/java/com/mobile/physiolink/ui/psf/PsfSettingsFragment.java			0.00000	0.00000	0.00000
CBO	2.0	2.0	2.0	2.00000	0.00000 %			
LCOM	3.0	3.0	3.0	3.00000	0.00000 %			
WMC	3.0	3.0	3.0	3.00000	0.00000 %			

Metric	Pivot File	Similar File 1	Similar File 2	Optimal Class	Diff	Interest in Average LOC	Interest in Hours	Interest in Dollars
DIT	0	0	0	0.00000	0.00000 %			

Figure 63: Interest PDF Descriptive

7 Case Studies: Apache Hive and Apache Kafka

Two case studies were performed on Apache Hive and Apache Kafka. In this section, a brief analysis will take place discussing some of the cluster results and manually evaluate how accurate or inaccurate the results of the model are in these large and complex projects.

7.1 Apache Hive

Apache Hive consists of 6126 files in total while the model identified 343 clusters, thus giving 17 files per cluster on average. For projects of this size, this indicates smaller clusters than the desired size given that we are trying to achieve the MVC clustering logic. The project is also multi-modular, meaning that some packages can be related with each other while others cannnot.

The cluster below correctly consists of classes contained in the same package hierarchy in the original project (org.apache.hive.service.cli), with no outliers. An interesting observation is that it does not contain all classes of the package. As we can see on the left image, it did so both as an error and as a refactoring suggestion. With MVC login in mind, classes like **HiveSQLException** should not be in a package about command line interface classes. However, **CLIService**, **CLIServiceClient** and **CLIServiceUtils** should be. This error can be attributed to the model's tendency to create large number of clusters with less classes. While this is not inherently bad since this clustering imposes that we should probably create a subpackage containing handle classes only, it still is difficult at first glance to determine whether the logic was applied correctly here (top level package or subpackage?).

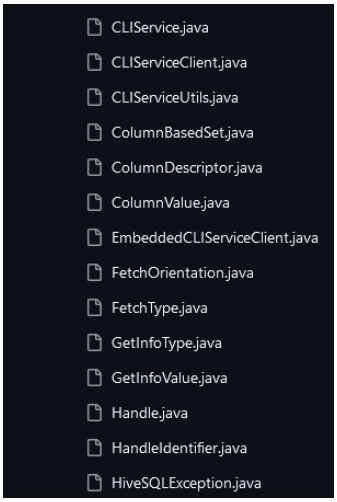


Figure 64: Apache Hive Handle Cluster

The following cluster consists of database classes, mainly ones that focus on drivers and connection management. It correctly groups the (org.apache.hadoop.hive.metastore.dataconnector.jdbc) except for **AbstractJDBCConnectorProvider**. Classes from the (org.apache.hadoop.hive.metastore.datasource) are also in the same group, indicating that the clustering model captured the database context of these classes. While not all classes inside (org.apache.hadoop.hive.metastore) are contained in this cluster, it still gives good insight as to what the database classes are.

Figure 65: Apache Hive DB Cluster

7.2 Apache Kafka

Apache Kafka consists of 3108 files in total while the model identified 214 clusters, thus giving 14 files per cluster on average. As we saw previously in Apache Hive, for projects of this size, this indicates smaller clusters than the desired size given that we are trying to achieve the MVC clustering logic. The project, just as Apache Hive, is also multi-modular, meaning that some packages can be related with each other while others cannot.

The following cluster consists of Response classes contained within the (org.apache.kafka.common.requests) package. It correctly differentiated between request and response classes, but as we can see in the project tree of the original project, both requests and response classes are in the same place (under requests). This could indicate a package refactoring

by putting the response classes in their corresponding package (org.apache.kafka.common.responses). Still, this is a minimal change that could be made, but it provides a good insight into how to properly separate classes based on the MVC pattern. It is worth mentioning that not all Response classes were grouped together. This is a common issue of the model as we will see with the next example.

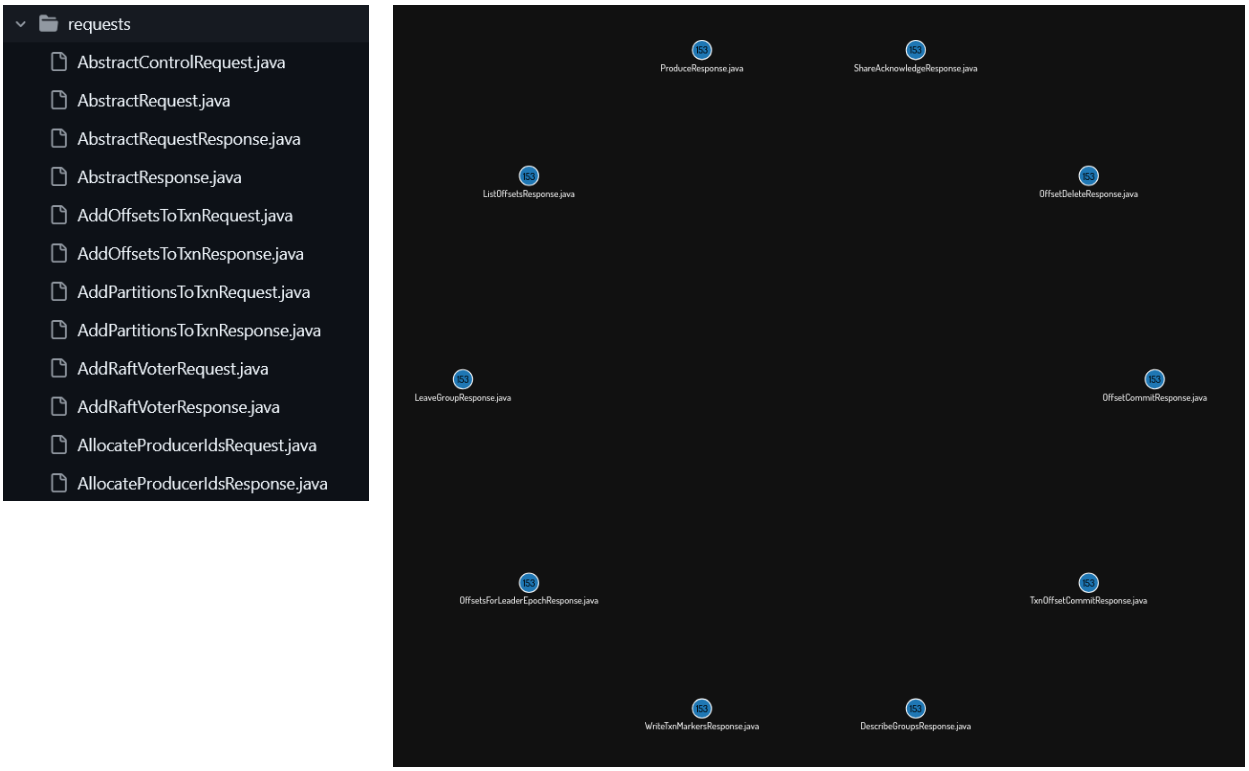
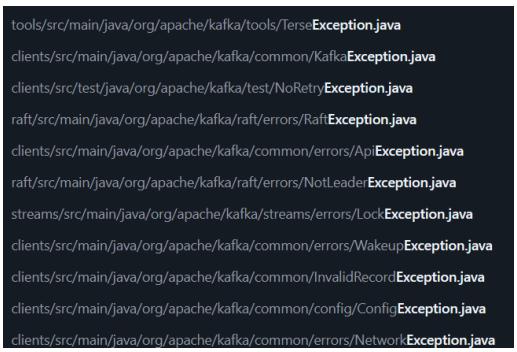


Figure 66: Apache Kafka Response Cluster

The following two clusters consist of Exception classes. While they both refer to the same type of classes, Cluster 2 contains more Exceptions than Cluster 1. Here we also see the same problem of clusters being split from each other while they both represent the same MVC logic. As we can see, the original project splits exceptions based on the domain they are used on (ConfigException is inside org.apache.kafka.common.config instead of org.apache.kafka.common.errors). This could also indicate a need for package refactoring if the user wishes to reorganize the project by MVC logic.



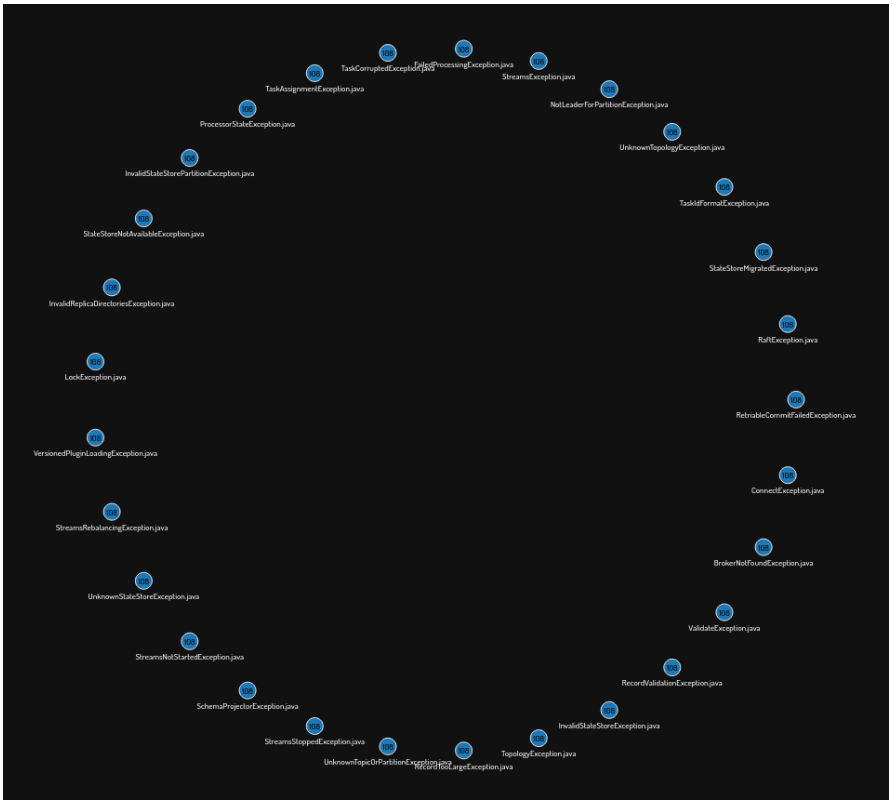


Figure 67: Apache Kafka Exception Cluster 1

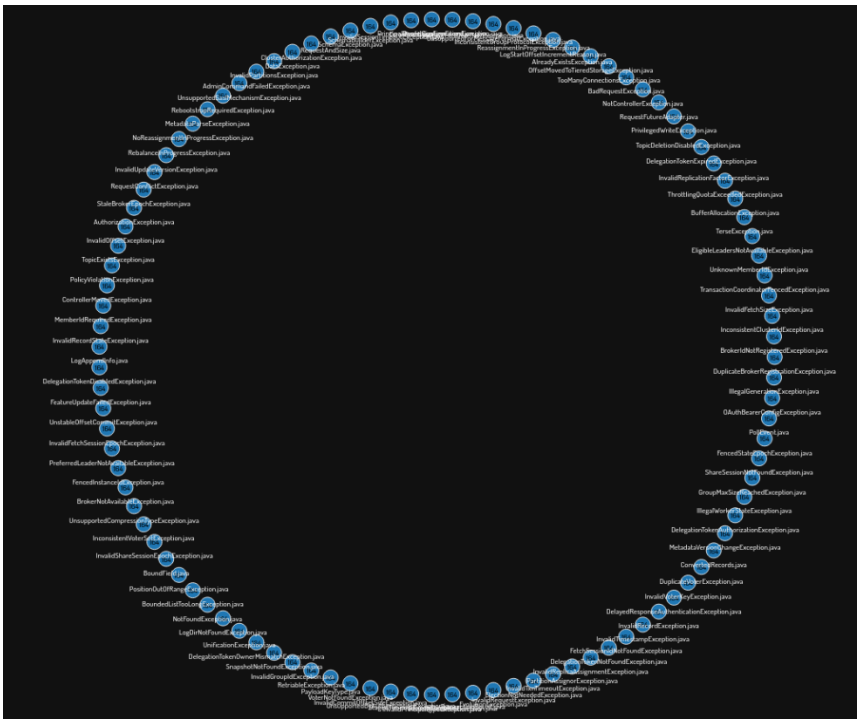


Figure 68: Apache Kafka Exception Cluster 2

8 Conclusion

While still in its initial phase of development, AstroCluster serves as a framework-agnostic solution to address the growing complexity of semantic analysis and technical debt management in modern software projects. Throughout this thesis, the challenges of clustering project code files based on their semantic relationships have been explored, particularly in diverse ecosystems comprising Spring Boot, Spring Framework, plain Java, and technical libraries like Apache Kafka and Apache Hive. By leveraging a fine-tuned embedding model, AstroCluster provides relatively satisfying performance for clustering project files, offering valuable insights into code organization acting as a "consultant" for the developer.

8.1 Improvements

First and foremost, the current MVC Java model must be fine-tuned further with a vast amount of data, and most importantly, diverse data. While UniXcoder supports many languages by default, the model can be run on other programming languages as well but that is not recommended. For this reason, the future development of new models must define two parameters: programming language and clustering paradigm (MVC or domain based). The development architecture was done in such a way that promotes and facilitates the addition or improvement of models.

Besides multi-language support and fine-tuning, further improvements can be made in the visualization of the results by adding a global search functionality for files and clusters, improving the user experience.

Furthermore, real-time analysis could be performed per commit or release alongside with custom Git configurations for private projects. This way, an evolution graph of the clusters and technical debt will be shown per unit.

9 Bibliography

[1] Allamanis, M., Brockschmidt, M., & Khademi, M. (2017). Learning to represent programs with graphs. *arXiv preprint arXiv:1711.00740*.

[2] Loshchilov, I. (2017). Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*.

[3] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., ... & Zhou, M. (2020). Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*.

[4] Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., & Yin, J. (2022). Unixcoder: Unified cross-modal pre-training for code representation. *arXiv preprint arXiv:2203.03850*.

[5] Ben-Nun, T., Jakobovits, A. S., & Hoefler, T. (2018). Neural code comprehension: A learnable representation of code semantics. *Advances in neural information processing systems, 31*.

[6] Frey, B. J., & Dueck, D. (2007). Clustering by passing messages between Data Points. *Science, 315*(5814), 972–976. https://doi.org/10.1126/science.1136800.

[7] Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., ... & Zhou, M. (2020). Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366*.

[8] Zhang, T., Ramakrishnan, R., & Livny, M. (1996). BIRCH: an efficient data clustering method for very large databases. *ACM sigmod record*, *25*(2), 103-114.

[9] Kenton, J. D. M. W. C., & Toutanova, L. K. (2019, June). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of naacL-HLT* (Vol. 1, p. 2).

[10] Rousseeuw, P. J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics*, *20*, 53-65.

[11] Caliński, T., & Harabasz, J. (1974). A dendrite method for cluster analysis. *Communications in Statistics-theory and Methods*, *3*(1), 1-27.s

[12] Davies, D. L., & Bouldin, D. W. (1979). A cluster separation measure. *IEEE transactions on pattern analysis and machine intelligence*, (2), 224-227.

[13] Bishop, C. M., & Nasrabadi, N. M. (2006). *Pattern recognition and machine learning* (Vol. 4, No. 4, p. 738). New York: springer.